

“Time-Series Driven Conversational Analytics for Understanding Temporal Chat Dynamics”

by

Sakshi Adhagale

2022-B-29032003

Suraj Bhadavkar

2023-B-08022005

Sanika Bari

2022-B-22052004C



AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

School of
Engineering

“Time-Series Driven Conversational Analytics for Understanding Temporal Chat Dynamics”

Thesis submitted in partial fulfilment
of the requirements of the degree of

Bachelor of Technology

in

Biomedical Engineering

by

Sakshi Adhagale

2022-B-29032003

Suraj Bhadavkar

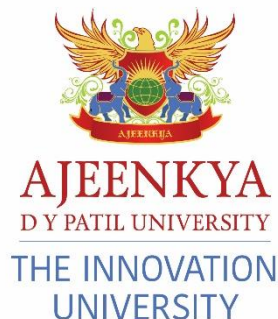
2023-B-08022005

Sanika Bari

2022-B-22052004C

Under the Supervision of

Prof . Aarti Kumari





AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

**School of
Engineering**

May 05, 2026

CERTIFICATE

This is to certify that the dissertation entitled “**Time-Series Driven Conversational Analytics for Understanding Temporal Chat Dynamics**”. is a bonafide work of **Sakshi Adhagale 2022-B-29032003 Suraj Bhadavkar 2023-B-08022005 Sanika Bari 2022-B-22052004C**

submitted to the School of Engineering, Ajeenkya D Y Patil University, Pune in partial fulfillment of the requirement for the award of the degree of “***Bachelor of Technology in Biomedical Engineering***”.

Prof. Aarti Kumari

Supervisor

Internal-Examiner/s

External Examiner

Dr. Prashant Kumbharkar

Dean-School of Engineering



AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

**School of
Engineering**

May 21, 2024

Supervisor's Certificate

This is to certify that the dissertation entitled “**Time-Series Driven Conversational Analytics for Understanding Temporal Chat Dynamics**”. submitted by **Sakshi Adhagale 2022-B-29032003** **Suraj Bhadavkar 2023-B-08022005** **Sanika Bari 2022-B-22052004C**

is a record of original work carried out by him/her under my supervision and guidance in partial fulfillment of the requirements of the degree of ***Bachelor of Technology in Biomedical Engineering*** at **School of Engineering, Ajeenkya DY Patil University, Pune, Maharashtra-412105**. Neither this dissertation nor any part of it has been submitted earlier for any degree or diploma to any institute or university in India or abroad.

Prof. Aarti Kumari

Supervisor



AJEENKYA
D Y PATIL UNIVERSITY
THE INNOVATION UNIVERSITY

School of
Engineering

Declaration of Originality

I, **Sakshi Adhagale 2022-B-29032003, Suraj Bhadavkar 2023-B-08022005 ,Sanika Bari 2022-B-22052004C** hereby declare that this dissertation entitled **“Time-Series Driven Conversational Analytics for Understanding Temporal Chat Dynamics”**. presents my original work carried out as a bachelor student of School of Engineering, Ajeenkya D Y Patil University, Pune, Maharashtra. To the best of my knowledge, this dissertation contains no material previously published or written by another person, nor any material presented by me for the award of any degree or diploma of Ajeenkya D Y Patil University, Pune or any other institution. Any contribution made to this research by others, with whom I have worked at Ajeenkya D Y Patil University, Pune or elsewhere, is explicitly acknowledged in the dissertation. Works of other authors cited in this dissertation have been duly acknowledged under the sections “Reference” or “Bibliography”. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission.

I am fully aware that in case of any non-compliance detected in future, the Academic Council of Ajeenkya D Y Patil University, Pune may withdraw the degree awarded to me on the basis of the present dissertation.

Place: Lohegaon, Pune

Sakshi Adhagale

Suraj Bhadavkar

Sanika Bari

Acknowledgement

We remain immensely obliged to **Prof. Aarti Kumari**, for providing me with the idea of this topic, and for his invaluable support in garnering resources for me either by way of information or computers also his guidance and supervision which made this Internship/Project happen. I would like to say that it has indeed been a fulfilling experience for working out this Internship/Project. I am also thankful to my institution and faculty members for their support, as well as my friends and peers for their cooperation and encouragement. Lastly, I extend my heartfelt thanks to my family for their constant motivation and support. This Internship/Project has been a truly enriching and fulfilling learning experience.

Sakshi Adhagale

Suraj Bhadavkar

Sanika Bari

ABSTRACT

With over 3.2 billion projected users by 2025 and more than 10 billion messages exchanged daily, messaging platforms have emerged as a dominant mode of global communication. However, tools for analyzing user behavior, communication patterns, and emotional context within these digital conversations remain limited. To address this gap, this paper presents the Conversational Time-coded Data Analyzer (CTDA), a novel application designed to extract structured, interactive insights from raw chat data through temporal and contextual analysis. Built using a Flask–React architecture and employing Plotly.js for dynamic visualizations, CTDA enables users to upload exported WhatsApp chat .txt files, which are then preprocessed and restructured into a clean DataFrame format. Feature extraction techniques are applied to analyze various aspects of the conversation, while integrated machine learning models perform sentiment analysis. The resulting insights are visualized to reveal individual and group-level communication patterns, behavioral dynamics over time, and temporal trends. CTDA provides these quantitative insights in an efficient, lightweight user experience without relying on external database storage.

Beyond simple message counting, the present work treats conversational records as time-indexed behavioural traces. This perspective is important because chat platforms preserve not only what participants say, but also when they speak, how quickly they respond, how frequently they return to a topic, and whether affective signals such as emojis, punctuation intensity, or abrupt silence coincide with shifts in participation. In practical environments such as project groups, classrooms, lab teams, and administrative channels, these temporal cues often reveal coordination quality more clearly than isolated text. A conversation that appears calm at sentence level may still reflect imbalance if one participant dominates the flow, if reply gaps widen around critical decisions, or if activity repeatedly collapses after short bursts. The CTDA framework therefore approaches chat analysis as a fusion of text mining, time-series reasoning, descriptive statistics, and interactive visualization. Rather than reducing conversations to static corpora, it reconstructs the evolving sequence of interaction so that user engagement, weekly activity cycles, sentiment drift, and collaborative intensity can be interpreted within their actual communication context.

The accompanying reference paper strengthens this approach by showing that temporal structure adds depth to conventional language analysis. Its discussion of clustered activity, uneven participation, sentiment progression, and dashboard-based interpretation supports the central thesis of this book: meaningful conversational analytics emerges when timestamps, participant behavior, and

message content are studied together. Accordingly, this project emphasizes end-to-end processing, beginning with raw WhatsApp exports and progressing through preprocessing, feature engineering, sentiment estimation, and visualization. The expected outcome is not merely a software artifact, but a decision-support system for understanding how digital groups function over time. Such a system can support research on collaboration, help educators review discussion health, assist organizations in identifying communication overload, and provide a foundation for future predictive or real-time analytics.

Keywords: Chat data analysis, Temporal data processing, Sentiment analysis, Conversational analytics, Behavioral pattern mining, WhatsApp data, Human-computer interaction, Natural language processing, Time-coded communication, Flask-React architecture, Data visualization, Social media analytics, Plotly.js, Lightweight data pipeline, User interaction analysis

CONTENTS

	PAGE NO.
CHAPTER 1: Introduction	
1.1 Introduction to project	12
1.2 Motivation	13
1.3 Project Aim and Objective	14
1.4 Existing system	15
CHAPTER 2 : Literature survey	
2.1 Related work	18
2.2 About Flask (Backend Framework)	25
2.3 About React.js (Frontend Framework)	26
2.4 About Plotly.js (Data Visualization Library)	27
2.5 About Natural Language Processing (NLP) and Sentiment Analysis Tools	28
2.6 About Pandas and Regex (Data Processing and Parsing)	29
CHAPTER 3: PROPOSED SYSTEM	
3.1 Conversational Time-Coded Data Analyzer (CTDA)	30
3.2 Purpose	31
3.3 Scope	31
3.4 Goals of the Proposed System	32
CHAPTER 4: Overview	
4.1 Overview of the Conversational Time-Coded Data Analyzer (CTDA)	34
4.2 Features	35
4.3 Technologies used	36
CHAPTER 5: System Requirements	

5.1	Hardware Requirement	37
5.2	Software Requirement	37
5.3	Functional Requirement	38
5.4	Non-functional Requirement	39
5.5	System Architecture	40
5.6	Use Case Diagram	42
5.7	Class Diagram	45
CHAPTER 6: System Implementation-Code Documentation		
6.1	Modules	47
6.2	Working of Modules	49
6.3	Backend Implementation	50
6.4	Frontend Implementation	53
6.5	Implementation and Workflow of CTDA Components	57
System Testing		
6.7	Testing	58
GUI Working Modules and Experimental Results		
6. 8	Final Output	59
Project Plan		
6.9	Project Plan	65
6.10	Gantt Chart	68
CHAPTER7: Analysis And Future Scope		
7.10	Analysis	73
7.11	Future Scope	75

LIST OF FIGURES

	Figure Name	Page No.
2.2.1	Flask	26
2.3.1	React.js	26
2.4.1	Plotly.js	27
2.5.1	NLP and Sentiment Analysis	28
2.6.1	Pandas and Regex	29
6.1.1	System Architecture	40
6.2.1	Use Case Diagram	43
6.3.1	Class Diagram	45
7.3.1	Raw data parsing	50
7.3.2	Preprocessed Data	51
7.3.3	Analytical Modules for calculating different results	51
7.3.4	REST API endpoints	53
7.4.1	File Upload Interface	54
7.4.2	Real-Time Feedback and Loading State	54
7.4.3	Data Visualization (Results)	55
7.4.4	Modular Components	56
9.1.1	User Interface	59
9.1.2	Chat Uploading and Parsing	60
9.1.3	Analysis Dashboard	61

LIST OF TABLE

	Table Name	Page No.
2.1.1	Literature Review	18
6.10.1	Gantt Chart	68

C

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

In the digital era, messaging platforms such as WhatsApp, Facebook Messenger, and iMessage have become integral to modern communication, facilitating the exchange of information, opinions, and discussions in both personal and professional contexts. These conversations encapsulate users' perceptions, attitudes, and behaviors, offering valuable insights into communication styles and interpersonal dynamics. Beyond linguistic content, these platforms also generate extensive temporal metadata—such as timestamps, read receipts, and last-seen indicators—which can reveal communication patterns, usage trends, and social engagement behaviors over time. These metadata elements play a crucial role in understanding the quality, frequency, and responsiveness of digital communication, particularly in group or professional settings. However, despite the ubiquity and richness of such data, systematic tools for analyzing these interactions, especially in a temporally aware and behavior-oriented manner, remain limited.

An established method for analyzing the structure and organization of interactions is Conversation Analysis (CA). Traditionally used in sociolinguistics, CA helps uncover social behaviors and engagement patterns embedded in communication. In this work, we present a temporal and behavior-focused adaptation of CA applied to professional online chat data through our proposed system, the Conversational Time-coded Data Analyzer (CTDA). CTDA explores how digital interaction patterns—rooted in both content and temporal metadata—can be linked to broader behavioral trends and user engagement. Unlike traditional linguistic analysis, CTDA emphasizes how language functions within everyday social interactions by using temporal markers as analytical anchors to uncover insights about communication timing, responsiveness, and interactional flow. For this study, WhatsApp is used as the primary data source due to its widespread adoption and the richness of its exported chat data, which includes unstructured textual content, emojis, media indicators, hyperlinks, and crucial timestamps. CTDA is capable of analyzing both individual and

group conversations to extract structured insights about user behavior, engagement patterns, and communication dynamics in digitally mediated environments. Ultimately, the system aims to assist researchers and organizations in understanding professional communication styles, identifying responsiveness bottlenecks, and improving team collaboration through data-driven insights.

From an academic perspective, the study of chat dynamics sits at the intersection of biomedical informatics, human-computer interaction, data science, and computational social analysis. Although conversational systems are often examined linguistically, time-aware analysis introduces a complementary quantitative lens. A burst of messages before a deadline, a sharp decline after a disagreement, or repeated late-night activity by a small subset of users can all indicate meaningful states such as urgency, dependency, coordination pressure, or disengagement. The introduction of time-series thinking into chat analytics therefore broadens the interpretive value of communication data. Instead of treating each message as an isolated line, CTDA examines conversation as a sequence of events distributed across time windows, allowing the system to distinguish routine traffic from notable deviations.

This framing is especially relevant in contemporary environments where many decisions, clarifications, and peer interactions occur in messaging applications rather than formal meetings. Professional and educational groups now leave behind rich conversational logs containing operational memory, behavioural signals, and emotional traces. However, these records are difficult to read manually once message counts rise into the thousands. The project therefore addresses a practical analytical gap: users need a way to move from raw, noisy exports toward interpretable evidence about participation, sentiment, tempo, and interaction balance. CTDA responds to this need by combining automated preprocessing with visual reasoning tools so that the user can see not only communication volume, but also communication rhythm.

1.2 Motivation:

The motivation behind this project stems from the growing need to analyze digital conversations beyond their textual content, particularly in professional and group communication contexts. With the rapid rise in messaging platform usage, large volumes of chat data are generated daily, yet most users and even organizations lack tools to extract meaningful behavioral insights from this unstructured data. While these platforms enable efficient and real-time communication, they also produce valuable temporal metadata that remains largely underutilized. This includes timestamps, response times, read

receipts, and activity patterns that, when analyzed systematically, can reveal deeper insights into communication behaviors, group dynamics, and engagement trends.

Our motivation lies in bridging this analytical gap by developing an application that allows users to upload exported chat data and extract structured insights without the need for complex tools or technical expertise. By applying methods rooted in Conversation Analysis and leveraging visual storytelling through data, the project aims to make digital interactions more transparent, measurable, and actionable. Additionally, with increasing reliance on messaging apps for professional collaboration, the ability to monitor patterns like participation frequency, conversation balance, and sentiment becomes valuable for individuals and organizations alike. CTDA seeks to empower users with a deeper understanding of their digital communication habits and promote healthy, productive interactions.

A further source of motivation comes from the fact that modern group communication is increasingly multimodal, fast-moving, and operationally important. Team decisions may be buried between casual remarks, while emotional temperature may be conveyed through abbreviated language, emoji patterns, and shifts in response delay rather than explicit statements. For researchers and practitioners, this means that the value of a chat archive lies not only in its textual content but in the structure of its unfolding. By designing CTDA around exported chat histories, the project aims to make large conversational datasets accessible to non-specialist users who need reliable summaries without performing manual coding or advanced statistical programming.

1.3 Project Aim and Objective:

○ Aim:

- The aim of this project is to develop the Conversational Time-Coded Data Analyzer (CTDA), a webbased application designed to analyze exported messaging group chats particularly from professional and educational contexts. The application aims to transform unstructured conversation data into structured, time-aware insights that highlight participant behavior, engagement levels, temporal trends, and thematic activity, thereby supporting more effective collaboration and outcome-driven communication.

○ Objectives:

- To enable users to upload and process exported WhatsApp group chat files containing time-stamped messages.

- To implement robust preprocessing techniques that clean and restructure raw textual data into analyzable formats.
- To apply feature extraction methods that identify patterns in message frequency, user activity, and engagement over time.
- To integrate machine learning models for sentiment analysis to detect emotional tones within conversations.
- To visualize extracted insights using interactive charts and graphs powered by Plotly.js, focusing on both individual and group-level metrics.
- To ensure the system operates without persistent data storage, maintaining user privacy and lightweight performance.

Taken together, the aim and objectives define a system that is both analytically useful and operationally realistic. The project does not attempt to solve every problem in computational discourse analysis; instead, it focuses on a clear and defensible problem space in which exported conversational data can be transformed into structured evidence. This objective-driven design is important for implementation, because each system component can be mapped to a measurable function: preprocessing improves data quality, feature extraction exposes behavior, sentiment analysis estimates emotional direction, and visualization improves interpretability. In this way, the objectives are not abstract academic statements but engineering requirements translated into software behavior.

The objectives also reflect several design trade-offs. A lightweight deployment model is preferred over a database-heavy architecture because many educational or exploratory use cases require quick analysis rather than long-term storage. Similarly, interpretable analytics are prioritized over opaque prediction-only systems, because end users need to understand why a trend appears on the dashboard. By structuring the project around clear objectives, CTDA becomes easier to evaluate: success can be judged through parsing accuracy, quality of insights, responsiveness of the interface, and usefulness of the generated timelines, charts, and participant summaries.

1.4 Existing System

Current methods for analyzing chat data range from manual review to automated tools with varying levels of sophistication. While some platforms offer built-in analytics focused on basic metrics, many rely on generic text analysis tools or manual inspection. However, these approaches often fall short in capturing the rich temporal and contextual dynamics inherent in messaging data, especially in informal, large-scale group chats.

○ **Manual Chat Review:**

- Users, researchers, or team leads often manually scroll through exported chat files to find relevant patterns or understand group dynamics.
- This process is time-consuming and inefficient, especially for large group chats with thousands of messages.
- Limited to surface-level observations, often missing deeper insights such as engagement frequency, sentiment shifts, or temporal trends.

○ **Basic Text Analysis Tools:**

- Generic text analysis tools can be applied to chat exports to extract word frequencies or generate simple word clouds.
- These tools lack contextual understanding and typically ignore the temporal aspect of conversation data.
- Sentiment analysis, if available, is often inaccurate when applied to informal chat language that includes emojis, slang, and shorthand.

○ **Platform-Integrated Analytics (Limited):**

- Some messaging platforms (e.g., Slack or Microsoft Teams) offer built-in analytics such as message counts or active hours.
- These tools are often subscription-based, platform-specific, and not available for universally used apps like WhatsApp.

A critical limitation of many existing approaches is that they either privilege convenience over depth or depth over usability. Manual review can sometimes capture conversational nuance, but it does not scale. Generic word-frequency tools scale better, yet they strip away sequence, participant balance, and timing. Platform analytics may provide convenient dashboards, but they are often closed, costly, or narrowly aligned to a single ecosystem. As a result, users who want to study exported conversations in a transparent manner frequently encounter fragmented workflows involving spreadsheets, scripts, and trial-and-error cleaning. CTDA is proposed as a response to this fragmentation by offering a unified path from raw chat file to interpretable output.

The need for a new system becomes even clearer when one considers informal language. Chat messages commonly include abbreviations, code-switching, emojis, forwarded media markers, and interrupted turn-taking. These characteristics reduce the effectiveness of tools designed for formal documents. A useful conversational analytics system must therefore respect the messiness of the input while still producing dependable structure. This is why CTDA places equal emphasis on preprocessing, temporal alignment, and visual explanation. Its contribution lies not only in measuring message counts, but in revealing communicative behavior that simpler tools routinely ignore.

C

CHAPTER 2

LITERATURE REVIEW

2.1 Related Work

Sr. No.	Title	Authors	Year	Overview & Approach (with Advantages & Limitations)
1	WhatsApp Wellbeing: A Study on WhatsApp Communication Quality and Stress	Nicole Blabst, Sarah Diefenbach (Ludwig Maximilians University, Munich)	2017	This paper explores the psychological impact of WhatsApp features such as Last Seen and Read Receipts. These time-related cues influence user behavior, response expectations, and social norms. It introduces the concept of a “saturation point,” where increased chat frequency leads to stress rather than meaningful communication. Advantages: Highlights behavioral effects of WhatsApp features, connects time-based metadata with communication quality, and introduces chat saturation concept. Limitations: Limited to WhatsApp platform, lacks strong empirical validation, and does not quantitatively measure saturation and communication depth.
2	Determining the Polarity and Statistics of Chat based on Sentiment Analysis	Piyush Anand, Raj Gupta (Galgotias University)	2022	This paper focuses on forensic analysis of WhatsApp data using tools like WhatsApp Explorer and WhatsApp Analyzer. It extracts structured and unstructured data (chats, media, contacts) to analyze communication patterns, including sentiment polarity, subjectivity, emoji usage, and interaction trends. Advantages: Demonstrates practical forensic tools, enables sentiment and behavioral analysis, and supports structured data examination. Limitations: Limited focus on ethical/privacy concerns, heavily tool-dependent, and applicability

				depends on device access and encryption constraints.
3	Forensic Analysis of WhatsApp Chat Data	Keshav Kaushik, Yash Katara (University of Petroleum and Energy Studies, Dehradun)	2022	This paper presents a system for analyzing WhatsApp chats using metrics such as word frequency, message count, media sharing, and links. It integrates sentiment analysis tools like TextBlob and VADER and uses visualizations such as heatmaps, word clouds, and timelines to identify trends and user behavior. Advantages: Combines text and sentiment analysis, strong visualization support, and useful across domains like workplace and social media analysis. Limitations: Limited dataset size, lacks real-time processing, and sentiment tools may fail with informal/contextual language.
4	Text Data Pre-Processing for Time-Series Modelling	Jitka Pomenkova, Petr Koráb, David Štrba (Brno University of Technology, Zeppelin University, Lentiamo Prague)	2023	This paper proposes a method to convert textual data into time-series format for econometric modeling. It applies sentiment analysis on long-term textual datasets (1933–2015) to study emotional and behavioral trends and their relationship with economic indicators like GDP. Advantages: Bridges text mining with time-series analysis, enables long-term sentiment tracking, and enhances economic modeling with qualitative data. Limitations: Complex preprocessing steps, requires large structured datasets, and may not generalize easily to real-time applications.
5	Post-disaster Situational Analysis from WhatsApp Group Chats of Emergency Response Providers	Pragna Saniul Somprakash Bandyopadhyay, Siuli Roy (IIEST Shibpur, IIM Calcutta, Heritage Institute of Technology Kolkata), Debnath, Haque	2016	This paper analyzes WhatsApp group chats for post-disaster needs assessment, focusing on NGO communications. It filters and clusters messages to identify regional resource gaps and improve decision-making. Tested on “Doctors For You” data from the 2015 Nepal earthquake and Chennai floods. Advantages: Provides actionable insights from real crisis data, demonstrates practical application, and highlights automated clustering benefits.

				Limitations: Manual analysis is time-consuming, clustering requires fine-tuning, and limited access to relevant WhatsApp data.
6	Text-based Conversation Analysis Techniques on Social Media using Statistical Methods	R. Sujatha, K. Nirmala (SRM Institute of Science and Technology)	2022	This paper explores social media conversation analysis using ML and DL techniques such as Naïve Bayes, SVM, Decision Trees, CNNs, and ensemble models. It follows a pipeline of extraction, preprocessing, feature engineering, and classification. Advantages: Integrates multiple ML approaches, provides end-to-end pipeline, and applicable to social and learning interactions. Limitations: Limited model-level evaluation, difficulty handling noisy data, and lacks real-time scalability discussion.
7	Sentiment Analysis of Mixed-Case Language using Natural Language Processing	Priyanka Awatramani, Rucha Daware, Hrushabh Singh Chouhan, Anmol Vaswani, Sujata Khedkar (VES Institute of Technology, Mumbai)	2021	This paper focuses on sentiment analysis of code-mixed language (Hinglish). It combines lexicon-based, rule-based, and machine learning methods such as SVM and Logistic Regression. A romanized Hindi lexicon is developed, achieving ~0.86 accuracy. Advantages: Addresses code-mixed language challenges, combines lexicon and ML techniques, and achieves strong performance. Limitations: Limited annotated datasets, multilingual complexity, and performance variation across dialects and informal text.

Table 2.1.1 Literature Review

The reviewed literature indicates that research on chat analytics generally falls into four overlapping strands: sentiment-oriented text analysis, temporal pattern mining, platform-specific activity dashboards, and broader conversation modeling. Earlier works have been effective in showing that exported chats can yield descriptive statistics such as most active users, common words, media counts, and polarity estimates. However, the literature also shows a recurring methodological gap. Many studies analyze messages as static text collections and therefore underutilize the chronological nature of conversation logs. When timing is included, it is often treated as a supplementary variable rather than a central analytic dimension. The present project advances the discussion by treating temporal behavior as a first-class feature of conversational understanding. In the digital era, messaging platforms such as WhatsApp, Facebook Messenger, and iMessage have become integral to modern communication, facilitating the exchange of information, opinions, and discussions in both personal and professional contexts. These conversations encapsulate users' perceptions, attitudes, and behaviors, offering valuable insights into communication styles and interpersonal dynamics. Beyond linguistic content, these platforms also generate extensive temporal metadata—such as timestamps, read receipts, and last-seen indicators—which can reveal communication patterns, usage trends, and social engagement behaviors over time. These metadata elements play a crucial role in understanding the quality, frequency, and responsiveness of digital communication, particularly in group or professional settings. However, despite the ubiquity and richness of such data, systematic tools for analyzing these interactions, especially in a temporally aware and behavior-oriented manner, remain limited. An established method for analyzing the structure and organization of interactions is Conversation Analysis (CA). Traditionally used in sociolinguistics, CA helps uncover social behaviors and engagement patterns embedded in communication. In this work, we present a temporal and behavior-focused adaptation of CA applied to professional online chat data through our proposed system, the Conversational Time-coded Data Analyzer (CTDA). CTDA explores how digital interaction patterns—rooted in both content and temporal metadata—can be linked to broader behavioral trends and user engagement. Unlike traditional linguistic analysis, CTDA emphasizes how language functions within everyday social interactions by using temporal markers as analytical anchors to uncover insights about communication timing, responsiveness, and interactional flow. For this study, WhatsApp is used as the primary data source due to its widespread adoption and the richness of its exported chat data, which includes unstructured textual content, emojis, media indicators, hyperlinks, and crucial timestamps. CTDA is capable of analyzing both individual and group conversations to extract structured insights about user behavior, engagement patterns, and communication dynamics in digitally mediated environments. Ultimately, the system aims to assist researchers and organizations in understanding professional communication styles, identifying responsiveness bottlenecks, and improving team collaboration through data-driven insights.

1.2 Motivation:

9 The motivation behind this project stems from the growing need to analyze digital conversations beyond their textual content, particularly in professional and group communication contexts. With the rapid rise in messaging platform usage, large volumes of chat data are generated daily, yet most users and even organizations lack tools to extract meaningful behavioral insights from this unstructured data. While these platforms enable efficient and real-time communication, they also produce valuable temporal metadata that remains largely underutilized. This includes timestamps, response times, read receipts, and activity patterns that, when analyzed systematically, can reveal deeper insights into communication behaviors, group dynamics, and engagement trends. Our motivation lies in bridging this analytical gap by developing an application that allows users to upload exported chat data and extract structured insights without the need for complex tools or technical expertise. By applying methods rooted in Conversation Analysis and leveraging visual storytelling through data, the project aims to make digital interactions more transparent, measurable, and actionable. Additionally, with increasing reliance on messaging apps for professional collaboration, the ability to monitor patterns like participation frequency, conversation balance, and sentiment becomes valuable for individuals and organizations alike. CTDA seeks to empower users with a deeper understanding of their digital communication habits and promote healthy, productive interactions.

A closer comparison of prior work also shows differences in analytical emphasis. Some studies prioritize polarity classification, producing useful but narrow emotional summaries. Others foreground dashboard analytics, providing descriptive counts without sufficient discussion of sequence or conversational dependency. A smaller group studies timing and interaction rhythms, but often without integrating accessible interfaces for non-programming users. CTDA draws from all three directions and attempts to unify them in a workflow that is both technically coherent and practically usable. This synthesis is important because chat behavior cannot be understood adequately from a single analytical dimension.

The reference paper provided with this work is particularly valuable because it connects message timing, user contribution, emoji use, sentiment progression, and dashboard visualization within one analytical workflow. Its reported observations—such as clustered activity during specific time windows, emotionally neutral baselines in professional discussions, and unequal contribution among participants—illustrate why time-aware analytics can reveal more than text inspection alone. This directly informs the thesis direction of CTDA. In the context of the literature survey, the main research gap can therefore be stated clearly: existing studies seldom integrate preprocessing, time-series interpretation, sentiment cues, and accessible visualization in a single lightweight application for exported chat data. CTDA is designed to address precisely that integration challenge.

2.2 About Flask (Backend Framework)

Flask is a lightweight Python web framework used to build the backend of CTDA. It offers flexibility, simplicity, and fine-grained control over the application's behavior. Flask allows the system to handle file uploads (e.g., WhatsApp chat exports), preprocess data, and expose REST APIs for the frontend to interact with. Being lightweight, Flask suits CTDA's goal of remaining database-free and efficient.

➤ Features Used:

- Routing and API development
- File handling and data parsing
- Integration with ML libraries like TextBlob or scikit-learn for sentiment analysis

Flask is particularly well suited to this project because the backend logic is organized around discrete analytical actions rather than long-lived enterprise transactions. File upload, parsing, participant selection, statistics retrieval, and chart generation can each be exposed as compact endpoints, allowing the frontend to request only the data required for a given view. This modularity improves maintainability and makes testing easier, since the developer can validate preprocessing, statistics, and visualization payloads independently. In addition, Flask allows the project to remain transparent: routing, middleware behavior, and serialization logic can be understood without the overhead of heavier frameworks.

Another reason for choosing Flask is its compatibility with the scientific Python ecosystem. Since the analytical pipeline depends on pandas, regular expressions, and NLP utilities, keeping the backend in Python reduces integration friction and enables a cleaner codebase. Rather than moving data between heterogeneous runtimes, the system can parse, analyze, and serialize results in one environment. This supports rapid prototyping, which is highly desirable in student research projects where the system architecture must remain understandable, reproducible, and extensible for future work.



Fig. 2.2.1 Flask

2.3 About React.js (Frontend Framework)

React.js is a modern JavaScript library for building fast and dynamic user interfaces. It enables CTDA to provide an interactive and responsive experience for users while displaying processed chat analytics. ●

Features Used:

- Component-based UI architecture
- React Router for navigation
- Integration with Plotly.js for chart visualizations

React.js complements the analytical goals of CTDA because conversational insight is not consumed in a linear report-like manner; it is explored interactively. Users typically want to move between overview statistics and participant-specific views, inspect multiple charts, filter users, and compare different activity patterns without reloading the entire interface. React's component model supports this style of exploration by separating upload, summary, chart, and error-handling logic into reusable units. As the system evolves, additional views such as comparative dashboards or export panels can be introduced without rewriting the full interface.

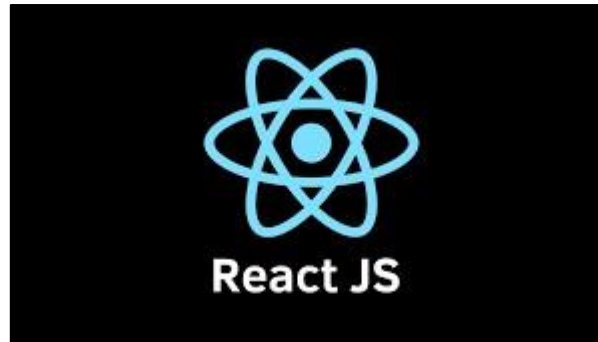


Fig. 2.3.1 React JS

2.4 About Plotly.js (Data Visualization Library)

Plotly.js is an open-source graphing library built on top of D3.js and stack.gl. It is used in CTDA to render rich, interactive charts that represent user engagement, message trends, activity heatmaps, and sentiment over time.

○ Advantages:

- Interactive and responsive charts
- Supports a wide range of visualizations: bar, line, scatter, heatmaps, pie charts ▫

Seamless integration with React

Plotly.js is central to the explanatory power of the system because temporal and behavioral patterns are often easier to recognize visually than numerically. A table of counts may confirm that one participant is highly active, but a time-based graph can show whether that activity is evenly distributed, concentrated around deadlines, or synchronized with specific peers. Likewise, a heatmap reveals weekly rhythm at a glance in a way that raw timestamps cannot. Interactive charts are therefore not decorative additions; they are analytical instruments that allow users to interpret structure, anomaly, and change over time.

The library is also advantageous because it supports a wide range of chart forms within a consistent rendering model. This matters when the same underlying dataset must support multiple narratives: participation distribution, sentiment trend, link sharing, message bursts, and active-hour intensity all require different visual encodings. Plotly's hover states, legends, zooming, and responsive layout improve the user's ability to inspect details without losing the overall pattern. In a thesis project centered on understanding dynamics, that ability to move between macro and micro views is especially valuable.



Fig. 2.4.1 Plotly.js

2.5 About Natural Language Processing (NLP) and Sentiment Analysis Tools

CTDA integrates NLP tools like TextBlob or VADER to analyze the emotional tone and polarity of messages. These tools help extract meaning from unstructured conversational text, supporting the sentiment analysis and topic trend modules in the system.

➤ Common Tasks Handled:

- Tokenization and cleaning of chat messages
- Sentiment scoring (positive, neutral, negative)

Natural Language Processing adds semantic depth to CTDA by converting otherwise opaque text streams into interpretable linguistic signals. In the context of chat data, NLP is useful not only for sentiment scoring but also for token filtering, keyword extraction, emoji-aware interpretation, and the detection of recurring conversational themes. However, conversational NLP differs from document NLP because chat language is compressed, colloquial, and context-sensitive. Messages are short, often fragmented, and frequently dependent on preceding turns. This means that lightweight sentiment tools must be applied carefully and interpreted as indicators rather than absolute truth values.

For this reason, the project frames sentiment output as one layer of evidence among several. A polarity score gains stronger meaning when read alongside message timing, sender dominance, and emoji intensity. For example, a spike in negative language may be analytically insignificant if it occurs in playful banter, but it becomes more meaningful when paired with longer response gaps or abrupt shifts in participation. The literature therefore supports a combined view in which NLP contributes valuable semantic cues, while temporal and behavioral measures protect against overinterpretation.

- Optional keyword or topic frequency extraction.

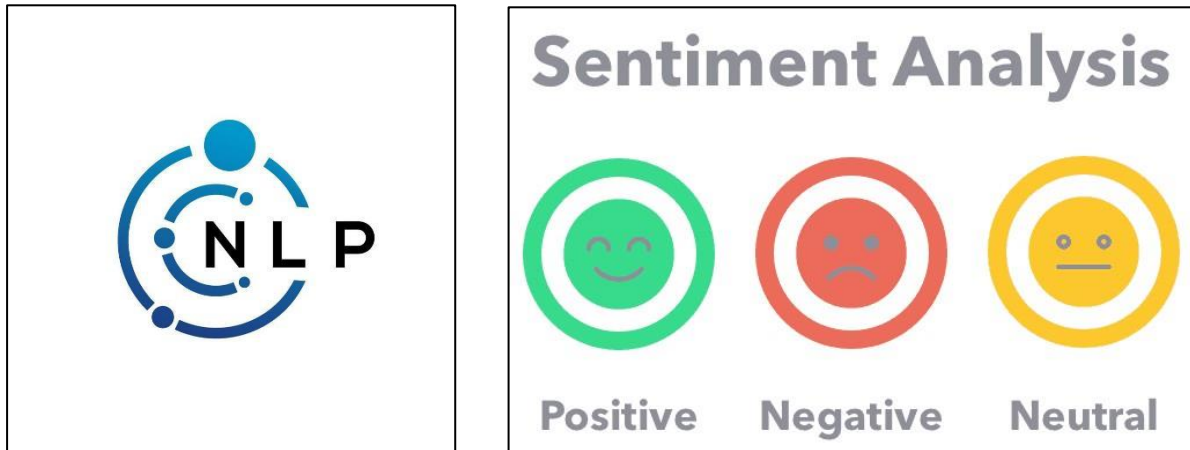


Fig. 2.5.1 NLP and Sentiment Analysis

2.6 About Pandas and Regex (Data Processing and Parsing)

The backend uses pandas to convert raw chat data into structured DataFrames for analysis. Regex (regular expressions) plays a vital role in parsing WhatsApp .txt files to extract timestamps, sender names, and message content accurately.

○ Functionality:

- Transforming unstructured .txt logs into a clean dataset
- Timestamping and sender attribution
- Supporting analytics modules by enabling efficient data slicing

Pandas and regular expressions form the backbone of data reliability in CTDA. Before any higher-level analytics can be trusted, the raw export must be converted into a structured representation in which dates, times, senders, messages, and derived fields are consistently encoded. Regex is essential for identifying the pattern boundaries that separate one message from another and for handling the specific formatting conventions of exported chat files. Pandas then provides the tabular abstraction required for filtering by user, aggregating by time interval, computing counts, and joining new analytical features such as sentiment labels or response intervals.

This combination is also a pragmatic engineering choice. Chat data analysis is iterative: once the initial frame is created, the developer often needs to test multiple transformations, compare subsets, and derive new metrics quickly. DataFrames support this exploratory workflow far better than manual string handling alone. In addition, pandas enables the system to remain interpretable during debugging, because intermediate

outputs can be inspected directly. For a thesis project, such transparency is a major advantage, since it strengthens methodological clarity and makes the implementation easier to explain and validate.



Fig. 2.6.1 Pandas and Regex

C

CHAPTER 3

PROPOSED SYSTEM

3.1 Conversational Time-Coded Data Analyzer (CTDA)

Problem Statement: “Develop a Conversational Time-Coded Data Analyzer (CTDA) to analyze professional messaging groups using chat data, providing actionable insights on participant engagement and topic trends to enhance collaboration and outcomes.”

CTDA is designed to process exported chat data from professional and educational messaging groups, transforming unstructured conversations into structured, time-coded information. By analyzing patterns of participant engagement, message frequency, and emerging topics over time, CTDA uncovers communication dynamics that are often missed in manual reviews. These insights help teams and organizations better understand interaction trends, identify key contributors, and monitor discussion focus, ultimately fostering improved collaboration and more meaningful, outcome-driven conversations.

The problem becomes more significant when one recognizes that digital groups are not simply channels for message exchange; they are environments in which coordination, support, conflict, and knowledge transfer occur continuously. A raw chat file preserves this activity in chronological form, but without analytical support it remains too dense for meaningful review. The CTDA problem statement therefore concerns information overload as much as algorithm design. The core challenge is to convert unstructured conversational records into evidence that can answer practical questions: Who drives the discussion? When does engagement peak? Does the emotional tone remain stable? Which users contribute consistently, and which appear only at critical moments?

By framing the problem in this way, the project moves beyond a generic chat analyzer and toward a focused research system. The emphasis on professional and educational groups is appropriate because these settings generate conversations with recognizable purpose, accountability, and rhythm. Such contexts make it easier to interpret participant activity as collaborative behavior rather than random social noise. Accordingly, CTDA is conceived as an analytical bridge between raw conversational traces and interpretable group-level insight.

3.2 Purpose

The purpose of this project is to design and develop an analytical tool that transforms unstructured chat data from professional or academic messaging groups into structured, time-based insights. CTDA is intended to assist team leaders, educators, and organizational stakeholders in monitoring communication effectiveness, understanding participation levels, and identifying key topics of discussion. It will automate the extraction of relevant patterns from large chat logs, thus eliminating the need for manual review and enabling faster, data-driven decision-making. The tool aims to enhance collaboration within groups by making visible the underlying conversational structure and highlighting opportunities for improvement.

The purpose of CTDA is therefore both descriptive and diagnostic. Descriptively, it summarizes what happened in a conversation across users and time windows. Diagnostically, it helps explain how interaction unfolded, where concentration or imbalance emerged, and whether the communication pattern reflects healthy collaboration or communication bottlenecks. This dual purpose makes the system useful for retrospective review, academic inquiry, and preliminary decision support. It is not intended to replace human interpretation, but to organize evidence so that human interpretation becomes more reliable and less time-consuming.

3.3 Scope

The project aims to develop a user-friendly web application that analyzes exported chat data to generate insightful visualizations and reports. Focusing initially on English WhatsApp chats in .txt format, the system will provide detailed dashboards and metrics at both individual and group levels.

The design emphasizes accuracy, performance, and extensibility to support future enhancements.

Project Scope:

- Develop a web-based application for uploading and analyzing exported chat data (e.g., WhatsApp).
- Process chat data to generate visualizations and reports on:
 - Participant activity and message distribution over time.
 - Trending topics and keyword frequency analysis.
 - Group-level engagement metrics and conversation heatmaps.
 - Interactive timelines showcasing communication flow.

- Provide individual and group-level dashboards for detailed insights.
- Focus on English-language chat exports in .txt format for the initial version.
- Design the system to be extensible for future support of additional chat formats and languages.
- Prioritize usability, system performance, and accuracy in delivering time-coded insights.

The scope defined for the current implementation is intentionally bounded. Focusing first on English-language WhatsApp exports allows the project to establish a stable analytical pipeline before generalizing to multilingual and cross-platform settings. Such scoping is methodologically sound because it reduces variability during system design and testing. At the same time, the stated scope remains broad enough to support meaningful real-world evaluation, since WhatsApp group chats are common in educational, organizational, and informal teamwork environments. The project therefore balances feasibility with relevance, a necessary condition for a successful undergraduate thesis.

Within this defined scope, several realistic application scenarios become possible. In academic groups, CTDA can help identify whether project communication is balanced or whether only a few students sustain the discussion. In workplace channels, it can reveal whether critical decisions are concentrated in narrow time windows or whether informational overload is created by a small number of highly active users. In research settings, the same pipeline can support exploratory studies of conversational rhythm, cooperation, and emotional tone. These applied scenarios justify the practical relevance of the project despite its intentionally limited initial domain.

- Assist teams in evaluating communication patterns and improving collaboration outcomes.

3.4 Goals of the Proposed System

➤ The goals of the CTDA system are:

Each goal of the proposed system can be interpreted as a contribution to analytical maturity. Upload and preprocessing goals ensure that the system can accept raw evidence. Statistical and NLP goals ensure that the evidence can be converted into behavioral and semantic indicators. Visualization goals ensure that these indicators become understandable to end users. Privacy and lightweight operation goals ensure that the solution remains practical in settings where users may be unwilling to store sensitive conversations on persistent servers. In combination, these goals define CTDA as a balanced system rather than a narrowly optimized prototype.

- Enable time-coded analysis of unstructured chat exports, transforming raw conversation logs into structured, visual insights.
- Quantify participant engagement by analyzing individual message contributions, reply patterns, and active periods.
- Identify trending topics and key themes by applying keyword extraction and frequency analysis techniques.
- Highlight communication gaps or inefficiencies within professional or academic group chats, supporting better collaboration practices.
- Provide data visualizations and summary reports that are easy to interpret and can be used by teams to assess and improve communication.
- Support decision-making by offering metrics and insights that can inform project coordination, leadership effectiveness, and group dynamics.

CHAPTER 4

OVERVIEW

4.1 Overview of the Conversational Time-Coded Data Analyzer (CTDA)

The Conversational Time-Coded Data Analyzer (CTDA) project is aimed at creating a robust analytical platform that processes unstructured conversational data from professional group messaging platforms, such as WhatsApp or Slack, and transforms it into meaningful insights. The platform is designed to help users understand and evaluate team dynamics, participant engagement, and topic trends through visual summaries and statistical analysis. Users will be able to upload exported chat transcripts, which the system will parse and analyze using natural language processing and time-based metrics to generate intuitive dashboards. These dashboards will reveal insights such as peak activity periods, individual user participation, recurring keywords, and trending discussion themes over time. The CTDA system seeks to offer professionals, educators, and teams a valuable tool for improving collaboration, identifying communication gaps, and making data-driven decisions based on actual conversation patterns.

At an overview level, CTDA can be understood as a staged interpretation engine. It begins by accepting an exported conversation, restructures that conversation into analyzable data, derives a set of statistical and linguistic measures, and finally presents the results through interactive visual forms. This staged view is important because it clarifies the dependency structure of the project: accurate insights depend on accurate parsing, and meaningful visualization depends on analytically coherent features. The project overview therefore reflects an architecture of cumulative refinement in which each stage improves the interpretability of the raw data.

The overview also highlights an important conceptual decision: CTDA is designed around retrospective conversational intelligence rather than live message intervention. This makes the system suitable for thesis-scale implementation while still leaving room for future real-time extensions. By focusing on uploaded exports, the project avoids unnecessary platform dependencies and concentrates on analytical correctness, privacy, and reproducibility. That choice is justified because strong retrospective analysis is a necessary foundation for any later predictive or streaming system.

4.2 Features

➤ The CTDA system will offer a variety of core features that facilitate easy exploration and understanding of conversational data:

- **Chat Data Upload:** Users can upload exported chat transcripts (e.g., WhatsApp .txt files), which the system will automatically parse and clean.
- **Participant Engagement Analysis:** The platform will calculate and visualize user-wise message frequency, participation trends, and identify the most and least active contributors.
- **Topic and Keyword Trends:** Using NLP techniques, the system will extract frequently discussed topics and trending keywords over time, helping users track the evolution of discussions.
- **Time-based Metrics and Heatmaps:** The analyzer will generate heatmaps and charts that display message volume by hour, day, or week to uncover peak activity windows and communication cycles.
- **Insight Dashboard:** A centralized, interactive dashboard will summarize key metrics such as total messages, engagement scores, conversation density, and active periods.
- **Downloadable Reports:** Users will be able to export summary reports for use in presentations or

internal documentation, enabling clear communication of findings.

These features are designed to provide users with a comprehensive, automated, and visual approach

The feature set is valuable because it supports multiple levels of reading. At the first level, users obtain immediate summary statistics such as message totals, active users, and high-level contribution patterns. At the second level, they can examine temporal charts to understand when activity rises or falls. At the third level, they can inspect linguistic and sentiment-oriented outputs to understand what kind of interaction accompanies those temporal shifts. This layered design prevents the dashboard from becoming either too shallow or too complex, and it mirrors the way analysts typically reason from overview toward detail.

A strong feature set also improves the system's practical relevance. In educational groups, for example, one instructor may care about participation balance, while another may care about bursts before deadlines. In workplace settings, a manager may care about active hours, turnaround patterns, or the prevalence of shared links and documents. By including multiple analytical views within the same workflow, CTDA becomes adaptable to several use cases without abandoning its central research identity.

to understanding complex chat interactions within group environments.

4.3 Technologies used

- **Python:** The core processing and backend logic of CTDA will be implemented in Python due to its rich ecosystem of libraries for data analysis (e.g., Pandas, NumPy), natural language processing (e.g., spaCy, NLTK), and backend development (e.g., Flask, FastAPI).
- **Flask:** A lightweight web framework used for building the backend APIs and managing chat file uploads, preprocessing, and response handling. Flask enables quick integration with frontend services and supports scalable microservices.
- **ReactJS:** The frontend of the application will be built using ReactJS to create dynamic and responsive user interfaces. React will be used to render interactive dashboards, visualizations, and chat metrics.
- **Plotly.js:** These JavaScript libraries will be used to implement interactive and visually appealing charts such as bar graphs, line charts, pie charts, and heatmaps for data visualization.
- **Tailwind CSS:** Tailwind CSS will be used for styling the user interface, ensuring a clean, modern, and responsive design.

Natural Language Toolkit (NLTK) / spaCy: These libraries will be employed to tokenize, filter, and analyze the conversational text for keyword extraction, frequency analysis, and topic modeling.

- The technology stack as a whole reflects careful alignment between system goals and implementation practicality. Python provides analytical depth, Flask offers lightweight service orchestration, React enables interactive delivery, and Plotly supplies expressive visualization. Tailwind CSS improves consistency and responsiveness in the interface, while NLP libraries enrich the semantic interpretation of text. Together, these technologies create a stack that is modern enough to support meaningful interaction yet simple enough to remain teachable and maintainable in an academic project setting.
- An additional advantage of the chosen stack is that each layer is supported by extensive documentation and community practice. This reduces development risk and makes the system easier to reproduce or extend by future students. For a black book, such reproducibility matters because the value of the project lies not only in the final dashboard but in the clarity with which the design can be communicated, defended, and improved.

C

CHAPTER 5

SYSTEM REQUIREMENTS

5.1 Hardware Requirements:

- A desktop, laptop, tablet, or smartphone capable of running a modern web browser (e.g., Chrome, Firefox, Edge, Safari).
- **Recommended minimum specifications:**
 - Processor: Dual-core CPU (Intel i3 or equivalent) or better
 - RAM: 4 GB or more
 - Storage: At least 100 MB of free space for browser cache and temporary files
 - Display: Minimum resolution of 1024x768 pixels for optimal UI experience
 - Reliable internet connection (minimum 1 Mbps recommended) to support smooth uploading and downloading of chat data and visualizations.

Although the hardware requirements are modest, their specification is still meaningful for system performance. Chat analysis may appear lightweight from a user perspective, yet the workflow involves file transfer, parsing, aggregation, and chart rendering. Adequate memory and browser capability are therefore important for maintaining responsiveness when large datasets are visualized. By keeping requirements low, the project aligns with the educational aim of accessibility, ensuring that the application can be used on commonly available devices rather than specialized infrastructure.

5.2 Software Requirements:

- Programming Languages: Python, JavaScript, HTML, CSS ➤ Libraries:
 - Python: Pandas, NumPy, Scikit-learn, NLTK, Matplotlib,
 - JavaScript: React.js (for front-end development) ➤ Frameworks:
 - Back-end: Flask (for server-side functionality) ▫ Front-end: React.js for building the user interface ➤
- Development Tools:
 - IDE: Visual Studio Code for development
 - Version Control: Git and GitHub for code management
 - Deployment: Vercel or Heroku for web application hosting

○ Environment: Jupyter Notebook for data analysis and testing

The software requirements reveal a workflow that integrates experimentation with deployment. Development tools such as Visual Studio Code and Git support structured software engineering, while Jupyter Notebook supports rapid testing of preprocessing logic, temporal aggregation, and exploratory analytics before those operations are formalized in application code. This combination is useful in research-oriented development because it allows algorithmic ideas to be validated interactively and then migrated into the production pipeline once their behavior is understood.

5.3 Functional Requirements:

○ Data Ingestion:

- Must support chat export files from WhatsApp in .txt format.
- Ability to parse and extract time-coded data (timestamps, sender names, message content).

○ Quantitative Analysis:

- The system must implement quantitative analysis techniques to analyze chat data, focusing on metrics such as message frequency, word count, and participant engagement levels.
- It should quantify interaction patterns, enabling insights into communication trends over time.
- Statistical methods should be employed to identify correlations between participant activity and conversation dynamics.

○ Statistical and Quantitative Analysis:

- Provide insights on conversation frequency, response times, and user participation.
- Quantify data on message count, most active participants, and dominant topics.

○ Visualization and Reporting:

- Provide interactive visualizations using dashboards (graphs, heatmaps, bar charts).

○ Machine Learning Integration:

- Predict communication trends and engagement levels based on chat data.

The functional requirements show that CTDA is expected to do more than generate descriptive charts. It must accept raw files robustly, construct a reliable analytical dataset, compute meaningful metrics, and present those metrics in a way that supports interpretation. Importantly, the requirements also introduce a forward-looking analytical ambition: the system should be extensible toward predictive views of engagement. Even if the current implementation emphasizes descriptive analytics, designing with predictive compatibility in mind encourages better feature engineering and data organization from the beginning.

Functionally, the system may also be understood as a progressive query environment. Even where explicit natural-language questioning is not yet implemented in full, the dashboard allows the user to ask analytical questions implicitly by switching scope and inspecting different visual summaries. Which user dominates the conversation? When does discussion intensity rise? Are links concentrated among a few participants? Does sentiment drift coincide with activity surges? The system's functions are therefore valuable because they support exploratory reasoning, not only one-time reporting.

These requirements also imply a strong need for correctness at each step of the pipeline. An error in timestamp parsing can distort temporal graphs; an error in sender identification can misrepresent participation; and inadequate text cleaning can contaminate sentiment or keyword outputs. Therefore, the functional specification is inseparable from data quality assurance. In practice, the success of CTDA depends not simply on having many features, but on ensuring that each feature is grounded in a trustworthy preprocessing pipeline.

5.4 Non-Functional Requirements:

○ Performance:

- The system must process datasets with up to 100,000 messages in less than 10 seconds.
- Ensure efficient data processing, especially during visual rendering.

○ Usability:

- The user interface should be simple, allowing non-technical users to easily upload chat data and view insights.

The non-functional requirements are equally important because conversational analytics systems are often judged by usability before they are judged by methodological sophistication. A technically powerful tool that produces slow, confusing, or unstable output will not be adopted in real settings. CTDA therefore emphasizes performance, clarity, and ease of use as core quality attributes rather than secondary refinements. This is an appropriate decision for a system meant to serve educators, students, and non-specialist users who need quick answers from complex data.

Privacy is another implicit non-functional requirement that deserves emphasis. Chat data frequently contains personal details, informal exchanges, and context-sensitive information. By avoiding unnecessary persistent storage and by processing uploaded files within a lightweight analytical workflow, the project reduces the data governance burden associated with conversational analysis. This design choice increases user trust and makes the system more appropriate for exploratory academic use.

5. SYSTEM ARCHITECTURE AND SPECIFICATION

5.5. System Architecture:

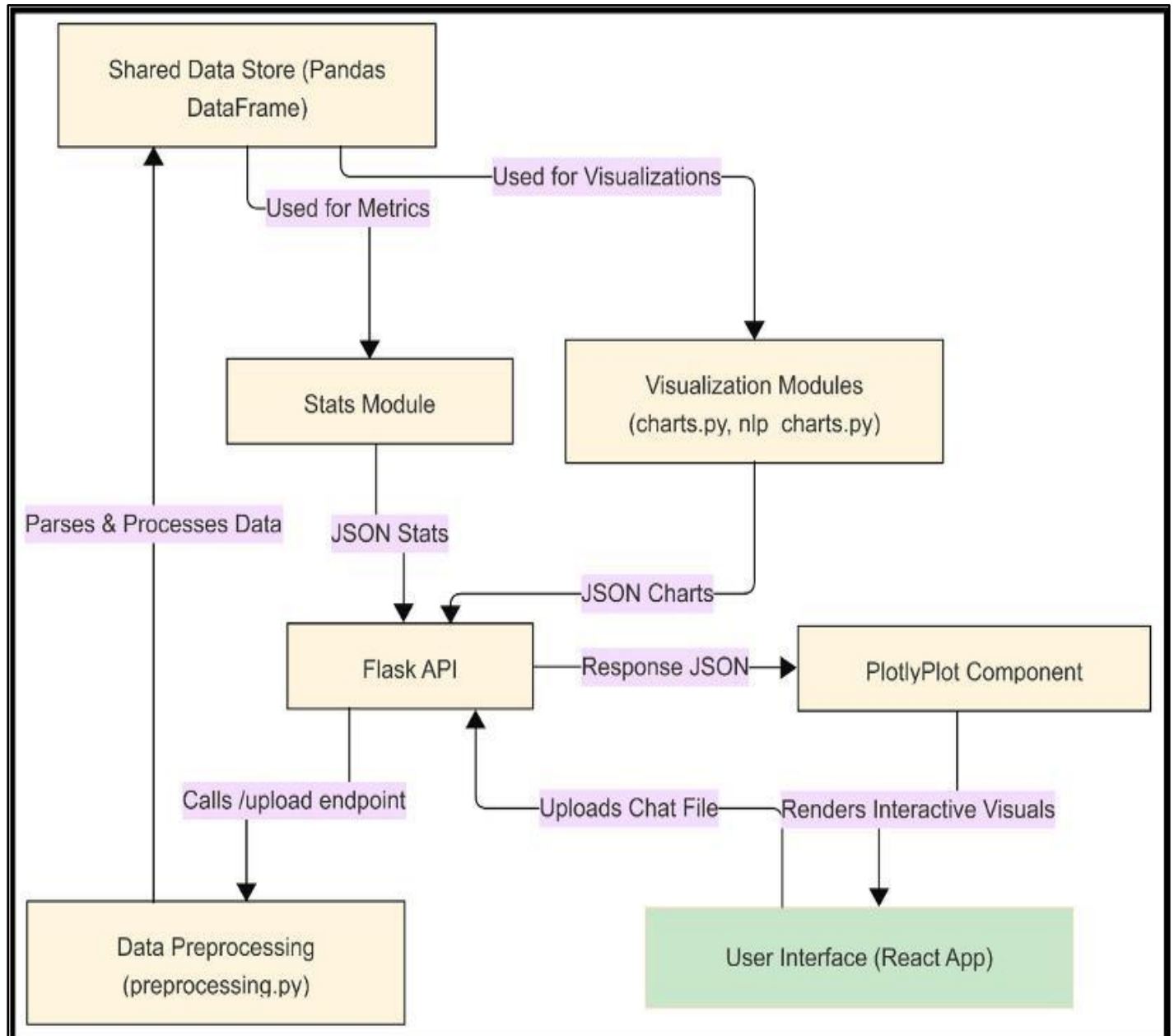


Fig 5.5.1 System Architecture

○ Purpose:

This architecture diagram provides a detailed look into the internal workings of the CTDA system, mapping out the flow of data from user input to final visualization. It highlights how various backend modules and frontend components interact to deliver analytical insights from WhatsApp chat data. This

diagram is especially valuable during design discussions and for technical documentation, showcasing the division of responsibility across layers.

○ **Actors:**

▫ **User**

○ **Role:** The primary actor interacting with the system via the frontend (React application).

○ **Responsibilities:**

○ Uploads WhatsApp chat text files. ○ Initiates analysis requests (e.g., sentiment, user metrics, trends). ○ Views and interprets visual results like charts and word clouds. ○ May filter data by individual users or reset sessions for fresh analysis.

• **System**

○ **Role:** Comprises the internal backend architecture that processes, analyzes, and visualizes data based on user input.

○ **Responsibilities:**

○ **Data Preprocessing Module:** Parses the uploaded chat file, structures data into a tabular format (Pandas DataFrame).
 ○ **Stats Module:** Computes key metrics such as total messages, media count, links, and active users.
 ○ **Visualization Modules (charts.py, nlp_charts.py):** Generates time-based plots, sentiment charts, and word clouds using Plotly.
 ○ **Flask API:** Serves as the communication bridge between frontend and backend, routing requests and returning JSON responses. ○ **Shared Data Store (Pandas DataFrame):** Acts as the central temporary data store accessed by multiple backend modules for consistent data flow.
 ○ **PlotlyPlot Component:** Renders the returned JSON visual data into interactive charts on the frontend for user interpretation.

The system architecture demonstrates a clear separation of concerns, which is essential for both correctness and maintainability. User interaction begins at the frontend, but the meaning of the application is produced in the backend analytical path. The uploaded file is first transformed into a structured data store, after which specialized modules compute descriptive, temporal, and sentiment-oriented features. The architecture is therefore not a mere technical arrangement; it encodes the logic of the research method itself. Each transition—from file to DataFrame, from DataFrame to metrics, and from metrics to visualization—corresponds to a stage of analytical interpretation.

Another architectural advantage lies in the way temporal granularity can be managed. Once the structured data store is available, the same conversation can be grouped by hour, day, week, or month

without redesigning the system. This makes the architecture analytically flexible: short-term bursts and long-term engagement drift can be studied using the same underlying pipeline. Such flexibility is especially important for conversational data, because meaningful patterns often appear only at the appropriate time scale.

The architecture is also consistent with privacy-aware computation. By relying on in-memory analytical state rather than mandatory persistent storage, CTDA minimizes retention of potentially sensitive conversation content. This does not eliminate all privacy concerns, but it does align the design with a principle of minimal data persistence. For student research and prototype deployment, that principle is both ethically appropriate and technically efficient.

This architecture also supports extensibility. Because preprocessing, statistics, NLP, and chart generation are logically separated, future improvements can be introduced with limited disruption. For example, a more advanced sentiment model, a network-analysis module, or a forecasting component could be attached to the same intermediate data representation. This modularity is one of the project's strongest architectural qualities because it ensures that the current thesis implementation can serve as a foundation rather than a closed endpoint.

5.2 Use Case Diagram:

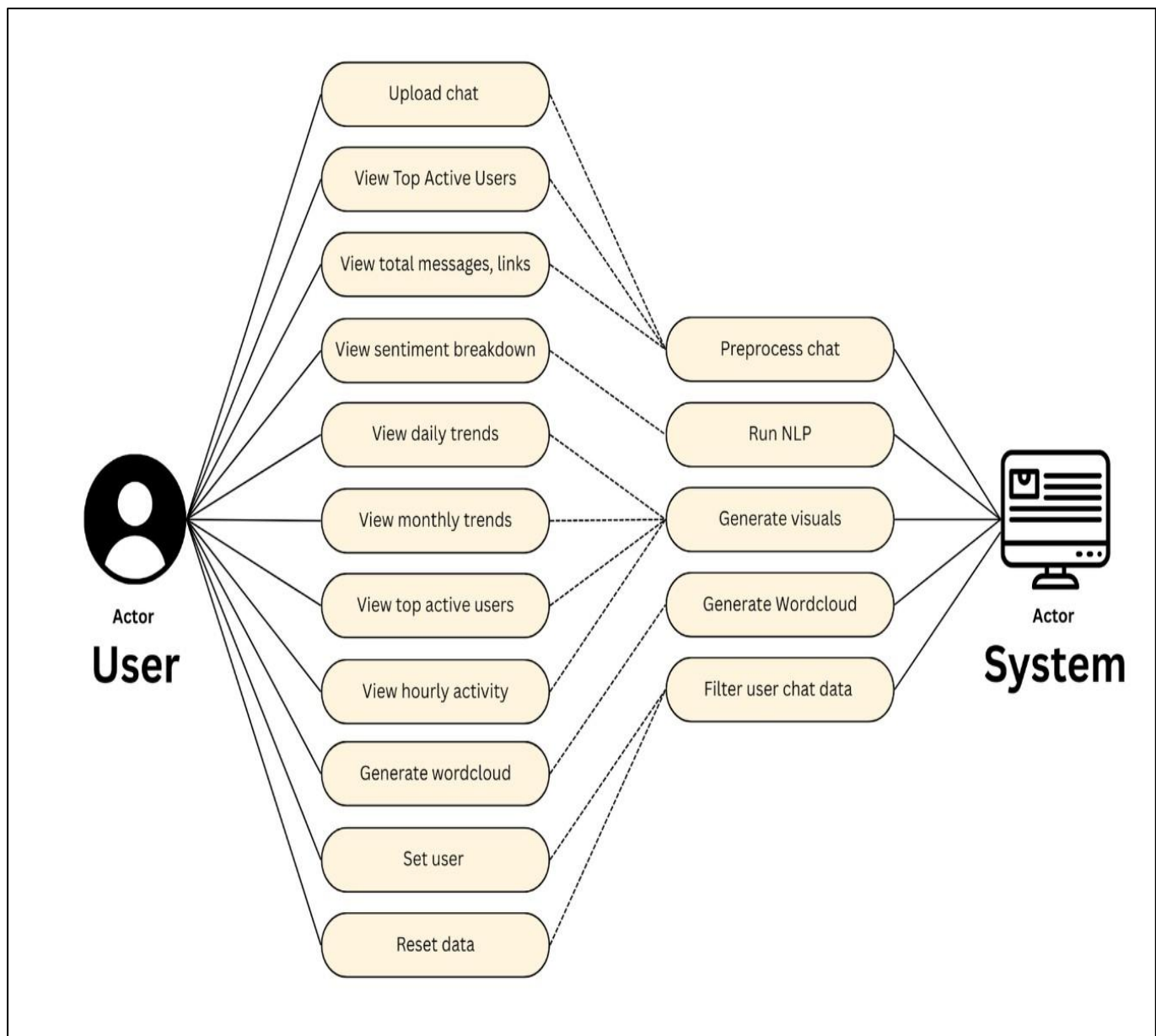


Fig 5.2.1 Use Case Diagram

○ Purpose:

This use case diagram illustrates the primary interactions between the end user and the system, emphasizing the main functionalities offered by the application and how they are executed through the backend modules. ➤ **Actors:**

- **User:** The person operating the application, typically uploading WhatsApp chat data and requesting visual/statistical analysis.
- **System:** The internal processing units (Flask backend, NLP engine, chart modules) that support the use cases triggered by the user.

○ Key Use Cases:

- Upload chat: The user uploads a WhatsApp .txt file for analysis.
- View Top Active Users: Displays the users contributing the most messages.

The use case perspective makes the system requirements concrete from the user's point of view. Instead of centering the discussion on internal code behavior, the use case view asks what meaningful actions the user expects to perform: upload data, select analytical scope, inspect statistics, compare participants, and interpret visual outputs. This perspective is especially useful in an academic project because it ties technical implementation to user value. Each backend route exists to satisfy a visible analytical intention, and each frontend element exists to support a specific interaction sequence.

- **View total messages, links: Shows overall statistical breakdown.**
- View sentiment breakdown: Classifies messages by sentiment (positive, neutral, negative).
- View trends (daily, monthly, hourly): Visualizes message activity over time.
- Generate Wordcloud: Creates a word cloud based on frequently used terms.
- Set user Reset data: Allows scoped analysis or resets the entire session.

○ **System Functions (on the right):**

- Preprocess chat: Converts raw chat into structured data (date, user, message).
- Run NLP: Applies sentiment analysis using VADER.
- Generate visuals: Builds Plotly charts and timelines.
- Generate Wordcloud: Specifically handles visual word cloud creation.
- Filter user chat data: Allows per-user filtering and scope setting.

5.3 Class Diagram:

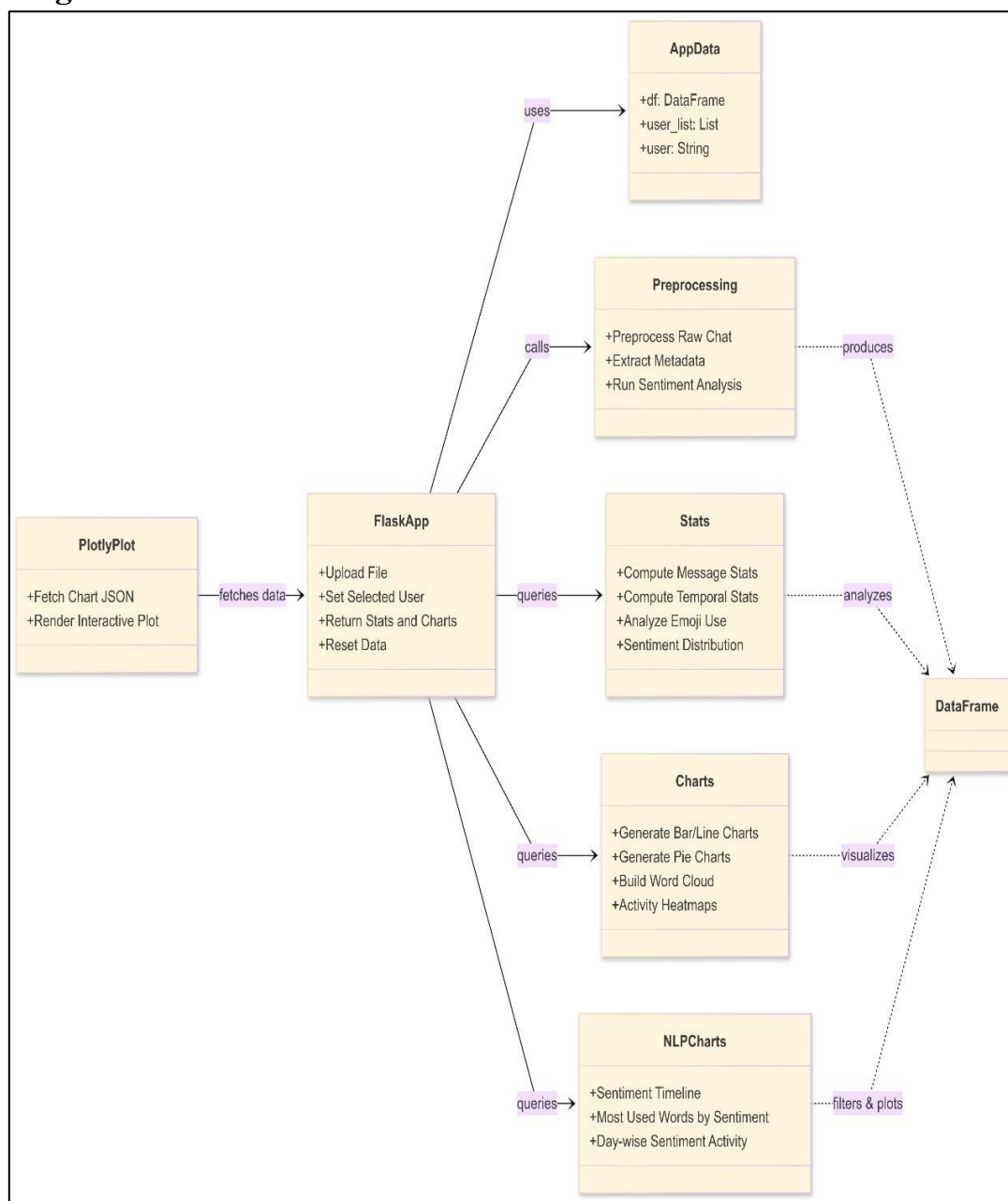


Fig 5.3.1 Class Diagram

Purpose:

This UML class diagram provides a high-level overview of the system's architecture, focusing on core classes, their responsibilities, and how they interact through data flow.

Main Components:

- **PlotlyPlot** (*Frontend React Component*):
 - Fetches chart JSON via HTTP.
 - Renders interactive Plotly graphs.
- **FlaskApp** (*Backend Controller*):
 - Exposes routes to upload data, set users, reset analysis, and serve chart/stat endpoints.

- Serves as the orchestrator for all processing.

- **AppData** (*In-memory Store*):

- Holds the processed DataFrame, current user, and user list across sessions.

- **Preprocessing:**

- Parses raw chat text.
 - Performs regex extraction, date conversion, user/message split.
- Computes VADER sentiment scores and tags rows.

- **Stats:** Computes core message stats like word counts, emojis, link sharing, temporal activity.

- **Charts:** Generates bar, line, pie charts using Plotly for things like message frequency and word usage.

- **NLP Charts:** Focuses on sentiment-related visualizations (e.g., sentiment timeline, most common sentiment words).

- **DataFrame:** Central data object flowing through all components, containing processed chat data.

The class-oriented description underscores the importance of data flow discipline in CTDA. The DataFrame functions as a shared analytical object from which multiple interpretations are derived. This arrangement reduces duplication and helps keep system behavior consistent across modules. A chart, a summary card, and a sentiment distribution should all be derived from the same processed state rather than from independently transformed copies. Such consistency improves debugging, strengthens trust in the outputs, and reflects sound system design.

A disciplined data-flow model additionally supports auditability. When each derived metric can be traced back to a known column, aggregation rule, or preprocessing step, the risk of silent analytical error is reduced. This is highly relevant in thesis work, where the system must not only function but also be explained convincingly during review or demonstration. Architectural clarity therefore contributes directly to academic defensibility.

CHAPTER 6

- SYSTEM IMPLEMENTATION-CODE DOCUMENTATION

6.1 Modules

The Conversational Time-Coded Data Analyzer (CTDA) is structured into multiple modules, each responsible for a specific set of functionalities ranging from file input to insight generation and frontend rendering. These modular components ensure maintainability, scalability, and separation of concerns within the system.

● Chat Input & Preprocessing Module:

This module handles the ingestion of raw chat data (primarily WhatsApp .txt exports). It includes:

- Text normalization
- Timestamp extraction

- Participant identification
- Message classification (media, system, or user message)

It ensures that the data is cleaned and parsed into a structured format suitable for analysis.

● Data Analysis Engine

Built with Python and Pandas, this is the core module of the system. It performs:

- Message frequency analysis (daily, weekly, monthly)
- Word count and activity tracking per participant
- Time-based patterns (e.g., most active hours)
- Sentiment or tone analysis (optional using TextBlob or VADER)
- Extraction of engagement metrics, like response time

This module outputs both raw metrics and derived insights to be rendered on the frontend.

○ User Profile & Insight Generation

This component creates summarized user profiles based on chat behavior, including:

- Top contributors

- Dominant times of activity
- Communication patterns (e.g., who replies to whom most)
- It converts numerical data into natural language insights that are easier for end-users to interpret.

○ Backend API Layer (Flask)

The Flask-based RESTful API serves as the communication bridge between the frontend and backend. It:

- Exposes endpoints for uploading and processing chat files
- Delivers processed data and visual summaries to the frontend
- Ensures secure, stateless communication

○ Visualization & Frontend (ReactJS)

This module provides a clean and interactive user interface. It is responsible for:

- Upload functionality and status tracking
- Displaying analytics using charting libraries like Chart.js or Recharts
- Showing user-specific and conversation-wide metrics
- Enabling users to filter, sort, and interpret insights dynamically

The module structure of CTDA reflects the practical reality that conversational analytics is a pipeline problem. No single algorithm can move directly from a raw text export to a credible dashboard. Instead, the system must first normalize and classify messages, then compute activity-based features, then generate user-level and conversation-level interpretations, and finally present the outputs in a coherent

interface. By assigning these tasks to separate modules, the project becomes easier to maintain and conceptually easier to explain in an academic setting.

Module-wise design also creates opportunities for evaluation. The preprocessing module can be tested for parsing accuracy, the analysis engine for metric correctness, the insight generator for consistency of summaries, and the frontend for proper rendering. This not only strengthens software quality but also supports the thesis requirement of demonstrating methodical implementation rather than ad hoc coding.

6.2 Working of Modules

When a user uploads a chat file, the Preprocessing Module is the first to run. It cleans and structures the raw input by parsing timestamps, extracting sender names, and isolating message content. System-generated messages (e.g., "Messages and calls are end-to-end encrypted") are also removed during this process.

Once the data is preprocessed, the User Generation Module scans through the data to identify all unique participants in the conversation. These user profiles are displayed on the frontend, allowing the user to select a specific participant for analysis. Alternatively, the user can choose to analyze the entire chat, which provides an aggregate analysis of the overall conversation, without focusing on any one participant.

After the user or overall chat selection, the Data Analysis Module processes the relevant data. The analysis includes:

- Total and average number of messages per participant
- Hourly and daily message distribution
- Response patterns across participants
- Word frequency and trending topics
- Top contributors based on the number of messages sent
- Most active or busy users, based on message frequency and engagement
- Temporal chat trends across the entire conversation or user

The Flask Backend Module manages the entire process behind the scenes, providing API endpoints for each stage of the workflow. The React Frontend Module communicates with the backend, fetching the processed results and visualizing them using dynamic, interactive graphs and charts.

The visuals are fully interactable, allowing users to explore the data in more depth by hovering over or clicking on specific elements within the graphs. Additionally, users have the option to store the generated results as a PNG picture, which can be downloaded for offline reference or reporting purposes.

These visualizations allow users to interpret communication dynamics, engagement patterns, and topic trends within the selected scope — whether that be an individual participant or the entire chat. The system is fully client-driven and stateless, meaning no data is stored persistently, and all processing occurs in session.

The working sequence of modules is significant because it mirrors the logic of evidence refinement. Raw text is inherently noisy and ambiguous, but each successive step reduces uncertainty. Participant extraction identifies the actors, time parsing places messages on a temporal axis, and feature computation converts observations into measurable indicators. Only after these steps does visual interpretation become trustworthy. This workflow-based understanding is essential for explaining why CTDA's outputs are analytically meaningful rather than merely decorative.

Another strength of the workflow is that it supports both whole-chat and participant-specific reasoning. Many group conversations appear balanced at aggregate level while hiding strong asymmetry at user level. By allowing the user to move between these scopes, CTDA helps uncover differences between group rhythm and individual behavior. That capability is especially useful when studying collaboration quality, response burden, or the emergence of dominant voices within a conversation.

6.3 Backend Implementation

The Conversational Time-Coded Data Analyzer (CTDA) relies on a lightweight and scalable backend architecture developed using Python and Flask. This backend provides essential services such as user interaction processing, chat data analysis, file handling, and communication with the frontend.

Key Backend Functions:

- **File Upload and Parsing:** Users upload WhatsApp .txt files through the frontend. The Flask server processes these files by reading line-by-line, identifying time-coded entries, and segmenting user messages.

	date	user	message	year	month
3	2022-01-04 15:46:00	Rachit Roshan, DYPCOE	<Media omitted>	2,022	January
4	2022-01-04 15:46:00	Rachit Roshan, DYPCOE	For online course, you can opt f	2,022	January
6	2022-01-04 21:02:00	Rachit Roshan, DYPCOE	wednesday SME Subject has	2,022	January
7	2022-01-04 21:02:00	Rachit Roshan, DYPCOE	10:00-11:00	2,022	January
8	2022-01-04 21:36:00	Sahil Sapkal, Dyp	This message was deleted	2,022	January
10	2022-01-04 22:09:00	Sir Hardik Mandwe, Dyp	This message was deleted	2,022	January
11	2022-01-04 22:09:00	Sir Hardik Mandwe, Dyp	This message was deleted	2,022	January
12	2022-01-04 22:10:00	Sir Hardik Mandwe, Dyp	You're invited to a Teams meeti	2,022	January

Fig 6.3.1 Raw data after parsing

- **Data Preprocessing:** The backend performs cleanup operations including removal of system messages, media notifications, and malformed entries. It also normalizes timestamps and extracts usernames for analysis.

message	year	month	month_	day	only_date	day_name	hour	minute	word_count	period	po	ne	nu	value
<Media omitted>	2,022	Januar	1	4	2022-01-04	Tuesday	15	46	2	15-16	0	0	1	1
For online course, you can opt f	2,022	Januar	1	4	2022-01-04	Tuesday	15	46	13	15-16	0	0	1	1
wednesday SME Subject has	2,022	Januar	1	4	2022-01-04	Tuesday	21	2	13	21-22	0	0	1	1
10:00-11:00	2,022	Januar	1	4	2022-01-04	Tuesday	21	2	1	21-22	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	21	36	4	21-22	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
You're invited to a Teams meeti	2,022	Januar	1	4	2022-01-04	Tuesday	22	10	22	22-23	0.116	0	0.884	1

Fig 6.3.2 Preprocessed data

- **Analytical Modules:** Once preprocessed, the data is analyzed using Python libraries (like pandas, NLTK, or spaCy) to compute metrics such as:
 - Message frequency over time
 - Word clouds and keyword extraction
 - Participant-wise message contribution
 - Sentiment analysis (if implemented)

```

26
27 > def top_users(df):
28
41 > def top_words(df, user):
42
64
65 > def timeline(df, user):
66
81
82 > def busiest_hours(df, user):
83
101
102 # days plotted against number of messages per day
103 > def busiest_days(df, user):
104
121
122 # days plotted against number of words(content length) per day
123 > def daily_wordcount(df, user):
124
141
142 # months plotted against number of messages per month
143 > def busiest_months(df, user):
144
161
162 # months plotted against number of words(content length) per month
163 > def monthly_wordcount(df, user):
164
181
182 > def activity_heatmap(df, user):
183
205
206 > def wordcloud(df, user):
207

```

Fig 6.3.3 Analytical Modules for calculating different results

- **REST API Endpoints:** Flask exposes a set of endpoints (e.g., /upload, /set-user, /top-stats, etc) that allow the React frontend to interact with the backend asynchronously and fetch results.

The backend implementation is where methodological intent becomes executable logic. Parsing routines transform conversational text into rows, preprocessing removes structural noise, analytical functions derive statistics, and API routes package the results for the frontend. Because the backend owns the validity of the analysis, its design must be disciplined and transparent. A well-structured backend prevents hidden inconsistency between views and makes it possible to trace each dashboard value back to a reproducible transformation.

Algorithmically, the backend can be viewed as a sequence of transformations. First, line-oriented parsing identifies message boundaries using export-format markers. Next, normalization resolves date formats, user labels, and message body inconsistencies. Feature derivation then adds behavioral variables such as word count, media indicators, URL presence, day and hour buckets, and sentiment labels. Finally, aggregation functions summarize these variables across the selected scope. This transformation view is useful because it explains why backend quality directly determines analytical validity.

Response-time style analysis, even when approximated, further enriches the backend's analytical potential. When timestamps are ordered correctly, the system can estimate conversational pace by examining gaps between consecutive messages or between turns from different users. Such measures provide insight into urgency, responsiveness, and interaction density. Although they require careful interpretation, they illustrate how temporal analytics can extend beyond raw message frequency into deeper behavioral characterization. From a technical standpoint, the backend also embodies several design trade-offs. Lightweight computation is favored over heavy infrastructure, in-memory processing is favored over persistent storage for privacy and simplicity, and clearly separated analytical routines are favored over monolithic code. These choices are appropriate for a research prototype that must remain understandable while still delivering meaningful output. They also make the system easier to benchmark, debug, and extend.

In future iterations, the backend could support richer analytics such as conversation segmentation, response-chain reconstruction, anomaly detection, or forecast-oriented trend modeling. The present design already prepares for such growth by organizing functionality into reusable analytical units. This is a strong implementation quality because it shows that the project was not only built to work, but built to evolve.

Development Stack and Implementation

The CTDA web application will be developed using the MERN stack (MongoDB, Express, React, Node.js), chosen for its robustness, scalability, and ease of integration.

1. **Frontend (ReactJS):** The user interface will be designed using ReactJS to display interactive charts and graphs for visualizing the results of the chat analysis. This will help users easily interpret the insights generated.

2. Backend (Node.js and Express): The backend will utilize Node.js with Express to handle API requests, manage data processing, and facilitate communication between the frontend and the database.
3. Database (MongoDB): MongoDB, a NoSQL database, will be used to store both processed chat data and analytical results. Its flexible structure will allow for easy scalability as the project evolves.

RESULTS AND DISCUSSION

1. For the target of performing temporal and statistical chat based analysis, Whatsapp chats prove to be the ideal source for input corpus as it contains rich, unstructured data, including text, emojis, multimedia (such as images and videos), timestamps, and more. This offers a complex, multi-dimensional data source for analysis. It is also the most trouble-free to acquire with user's consent.
2. WhatsApp provides both personal (one-on-one) and group chat interactions, making it suitable for analyzing different dynamics in communication, such as social behavior in groups versus individual exchanges.
3. Use of Traditional Machine Learning approaches is best suited for text classification and statistical analysis, whereas Deep Learning approaches facilitate linguistic analysis.
4. The use of visualizations like word clouds, sentiment analysis charts, and message volume graphs allows for a clearer understanding of communication patterns, sentiment distribution, and user engagement within the WhatsApp group.

```

42 @app.route('/upload', methods=['POST'])
43 > def upload_file():...
58
59
60 @app.route('/set-user', methods=['POST'])
61 > def set_user():...
68
69
70 # 1 : numerical stats - num of messages, links etc
71 @app.route('/top-stats')
72 > def serve_top_stats():...
77
78
79 # 2 : temporal stats - busiest day, week, month
80 @app.route('/temporal-stats')
81 > def serve_temporal_stats():...
85
86
87 # 3 : dataframe of users and their chat contribution in %
88 @app.route('/contributions')
89 > def serve_contributions():...
94
95
96 # 4 : bar graph of top 5 users with message counts
97 @app.route('/top-users')
98 > def serve_buzy_users():...
102
103
104 # 5 : horizontal bar graph of top 10 (or less) most common words with occurrence
105 @app.route('/top-words')
106 > def serve_top_words():...

```

Fig 6.3.4 REST API endpoints

6.4 Frontend Implementation

The frontend of the Conversational Time-Coded Data Analyzer (CTDA) is developed using React.js, a modern JavaScript library known for building responsive and modular user interfaces. The UI is designed with a focus on simplicity, ease of navigation, and real-time feedback to the user.

Key Features of the Frontend:

- File Upload Interface:** Users are provided with a clean interface to upload exported WhatsApp chat .txt files. The file is validated before being sent to the backend for processing.

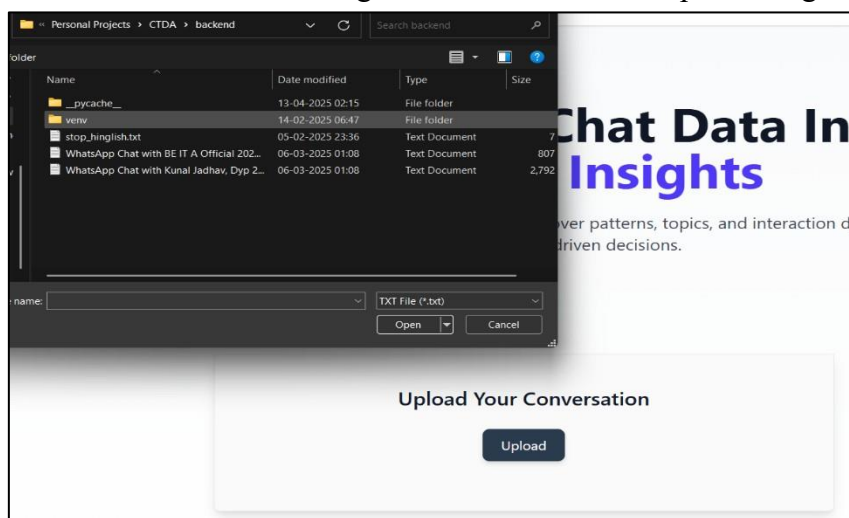


Fig. 6.4.1 File Upload Interface

- Real-Time Feedback and Loading State:** During file upload and data analysis, users see an animated loading screen to improve UX and indicate that processing is underway.

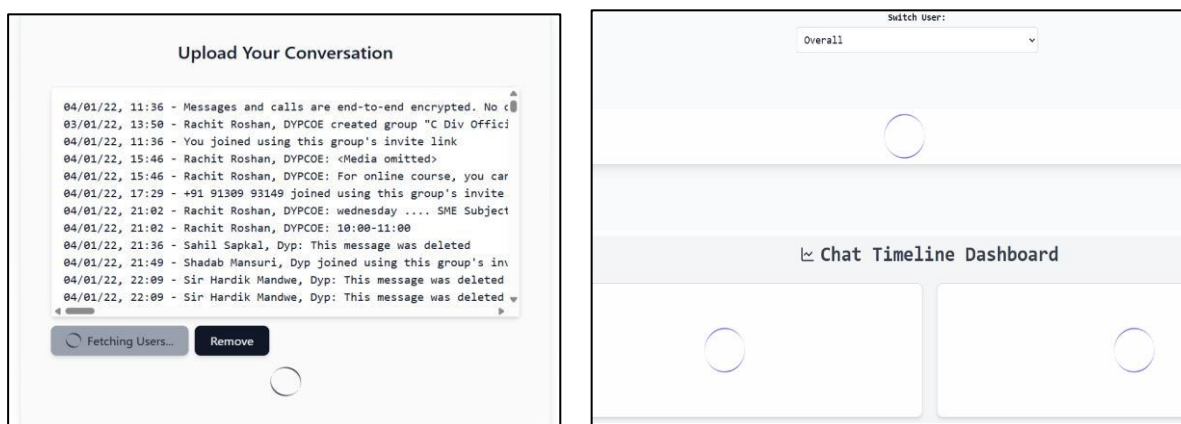


Fig 6.4.2 Real-Time Feedback and Loading State

- Data Visualization: The frontend receives analysis results in JSON format and dynamically renders visual insights such as:

- Bar and line charts for message volume over time
- Pie charts for user contributions
- Word clouds for keyword prominence

Tables for summary statistics

Plotly.js library is used for visualization.

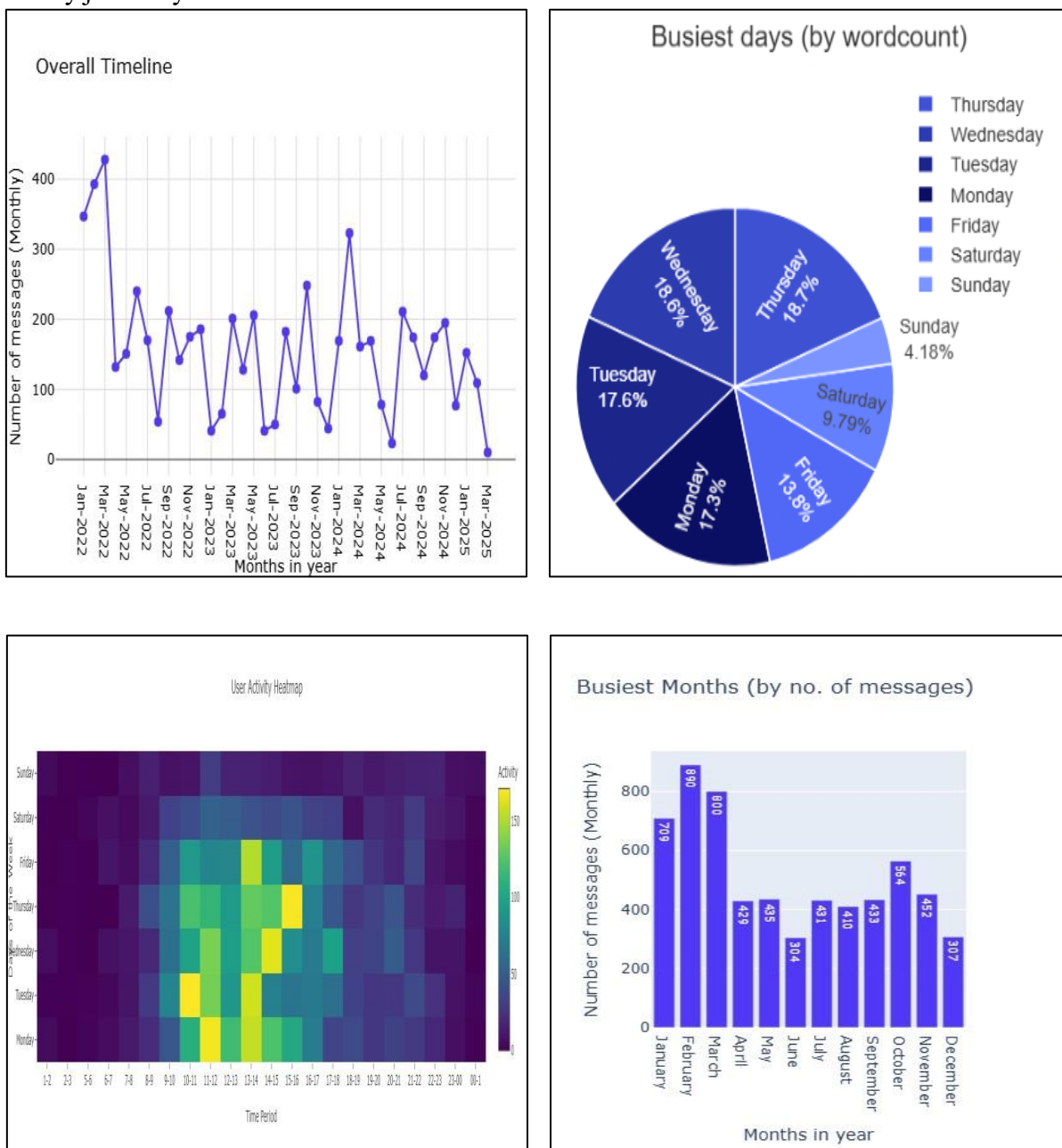


Fig 6.4.3 Data Visualizations (Results)

- **Modular Components:** The React codebase is componentized for better readability and scalability:
 - `Upload.jsx` – handles file input and form submission
 - `Results.jsx` – renders graphs and insights
 - `LoadingPage.jsx` – animated screen shown during processing
 - `ErrorComponent.jsx` – displays validation or server-side errors

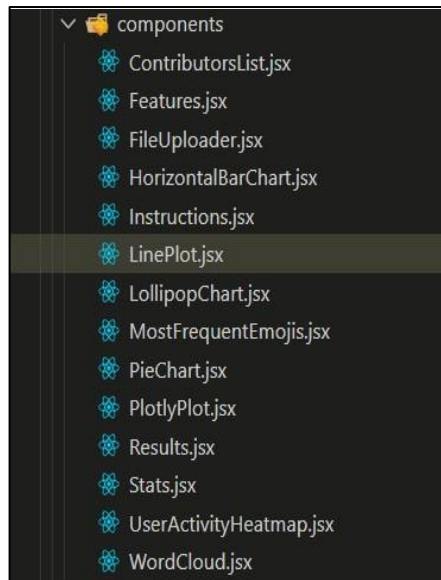


Fig 7.4.4 Modular Components

- **Styling and Responsiveness:** The UI is styled using Tailwind CSS, offering a clean, consistent design. The layout is responsive and mobile-friendly, ensuring accessibility across devices.
- **Interaction with Backend:** All communication between the frontend and backend is done via the Fetch API. Proper error handling and response parsing ensure a seamless user experience.

Frontend implementation is not merely a presentation concern in CTDA; it directly shapes the interpretability of results. Conversational analytics produces multi-dimensional output, and a poorly designed interface can obscure relationships that the underlying analytics correctly identify. The React frontend therefore plays a methodological role by sequencing the user experience: first establishing the upload context, then presenting summary findings, and finally enabling deeper inspection through visual modules. This progression helps users form an accurate mental model of the data.

The frontend also supports cognitive economy. Well-ordered summary cards, charts, and participant selectors reduce the amount of mental effort required to move from raw evidence to interpretation. This is a subtle but important quality in analytics systems, since users often abandon tools that appear informationally rich but cognitively exhausting. CTDA's interface design therefore contributes to the usability of the research method itself.

The frontend also embodies practical design choices related to responsiveness and clarity. Loading states are important because preprocessing and chart generation may take noticeable time for large datasets. Validation and error messaging are equally important because chat exports often contain formatting irregularities. By addressing these interaction details, the interface reduces user confusion and supports trustworthy use of the system. A thesis project concerned with applied analytics must account for such usability factors, since real analytical value depends on effective delivery.

6.5 Integration and Workflow of CTDA Components

The system is modular, ensuring separation of concerns and ease of development and maintenance. The major integrated components are:

➤ **React Frontend:**

- Handles file uploads
- Displays visual insights (charts, tables, downloadable summaries)
- Fetches data from the backend via API requests ➤ **Flask**

Backend:

- Parses uploaded chat files
- Extracts and analyzes data ▫ Returns results in JSON format ➤ **Visualization Module:**

- Uses Plotly.js in React
- Displays dynamic graphs such as daily message count, user activity, word clouds, etc.

The integration layer is what allows CTDA to function as a coherent analytical system rather than as a collection of unrelated scripts. Data produced in the backend must be serialized consistently, consumed correctly by the frontend, and rendered in forms that match user expectations. This coordination requires attention to payload structure, route semantics, state management, and chart configuration. In many student projects, integration is where promising modules fail to become a usable application. CTDA explicitly addresses this risk by emphasizing workflow continuity from upload to interpretation.

A well-integrated workflow also improves analytical reproducibility. When the same uploaded file reliably produces the same participant lists, summary statistics, and charts, the system becomes useful not only for demonstration but for repeated study. This consistency is essential if the project is later

extended to comparative analysis across multiple chat groups or time periods. Integration quality therefore has direct research value, not just software-engineering value.

6.6- SYSTEM TESTING

6.1 Testing

System testing is a critical part of ensuring that all components of the Conversational Time-Coded Data Analyzer (CTDA) work as expected. The following testing procedures were conducted to ensure the app performs well, is user-friendly, and provides accurate results:

○ Functional Testing:

- File Upload Verification: Ensure that the file upload functionality accurately accepts .txt chat files and validates their content before sending them for analysis.
- Chat Parsing Accuracy: Verify that the app can parse WhatsApp chat data correctly, extracting messages, timestamps, and user identifiers, while ignoring irrelevant data like metadata.
- Data Analysis Validation: Test that the system correctly analyzes the chat data and generates accurate insights, such as message frequency, user participation, and keywords.
- Results Display Verification: Ensure that the results, such as graphs, charts, and tables, are displayed correctly and contain relevant information that is easy to interpret.

○ Integration Testing:

- Backend Communication: Verify that the frontend correctly communicates with the Flaskbased backend. Ensure that API calls for uploading files, retrieving analysis results, and generating reports work as expected.
- Visualization Integration: Test the integration of data visualization libraries (like Recharts or Chart.js) to ensure that charts and graphs update correctly with the results from the backend.

○ Performance Testing:

- Load Testing: Simulate multiple users uploading chat files and analyzing them concurrently to ensure the app can handle high traffic and large datasets without crashing or slowing down.

- **Response Time Validation:** Test how quickly the application responds to queries, including the time taken for file upload, data parsing, analysis generation, and report rendering.
- **Stability Testing:** Verify that the app remains responsive and stable during peak usage periods, particularly during simultaneous data uploads and analysis requests.

○ **Usability Testing:**

- **User Interface (UI) and Experience (UX) Testing:** Conduct usability testing with representative users to assess the intuitiveness and effectiveness of the UI. Gather feedback on the ease of navigation, clarity of information, and overall satisfaction with the interface.
- **Accessibility:** Ensure that the application is accessible on multiple devices (desktops, tablets, and smartphones) and across various screen sizes. Check that the design remains responsive and visually appealing.
- **User Feedback and Improvement:** Collect feedback from users to identify pain points or areas for improvement. Based on the findings, refine the design and functionality to enhance user experience.

6.2 - GUI WORKING MODULES AND EXPERIMENTAL RESULTS

6.1 Final Output

○ **User Interface:**

- Clean and user-friendly interface designed for ease of use.
- Navbar-based navigation for quick access to various features like File Upload, Instructions, and Features
- Prominent upload section to allow users to import WhatsApp .txt files directly.
- Responsive design optimized for both desktops and tablets.

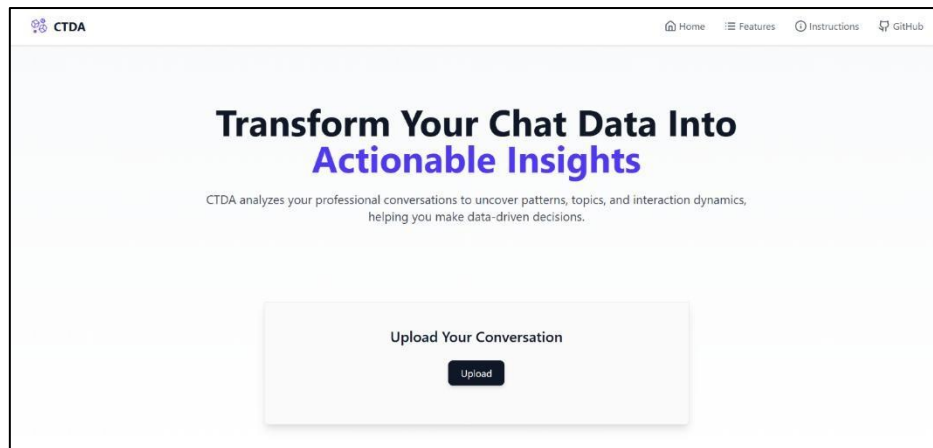


Fig 6.2.1 User Interface

○ Chat File Upload and Parsing:

- Upload feature for importing exported WhatsApp conversations in .txt format.
- Parsing mechanism automatically identifies:
 - Timestamps
 - Senders (users)
 - Message content

Upload Your Conversation														
04/01/22, 11:36 - Messages and calls are end-to-end encrypted. No c 03/01/22, 13:50 - Rachit Roshan, DYPCOE created group "C Div Offici 04/01/22, 11:36 - You joined using this group's invite link 04/01/22, 15:46 - Rachit Roshan, DYPCOE: <Media omitted> 04/01/22, 15:46 - Rachit Roshan, DYPCOE: For online course, you car 04/01/22, 17:29 - +91 91309 93149 joined using this group's invite 04/01/22, 21:02 - Rachit Roshan, DYPCOE: wednesday SME Subject 04/01/22, 21:02 - Rachit Roshan, DYPCOE: 10:00-11:00 04/01/22, 21:36 - Sahil Sapkal, Dyp: This message was deleted 04/01/22, 21:49 - Shadab Mansuri, Dyp joined using this group's in 04/01/22, 22:09 - Sir Hardik Mandwe, Dyp: This message was deleted 04/01/22, 22:09 - Sir Hardik Mandwe, Dyp: This message was deleted														
Get Users Remove														

message	year	month	month	day	only_date	day_name	hour	minute	word_count	period	po	ne	nu	value
<Media omitted>	2,002	January	1	4	2022-01-04	Tuesday	15	46	2	15-16	0	0	1	1
For online course, you can opt f	2,002	January	1	4	2022-01-04	Tuesday	15	46	13	15-16	0	0	1	1
wednesday ... SME Subject has	2,002	January	1	4	2022-01-04	Tuesday	21	2	13	21-22	0	0	1	1
10:00-11:00	2,002	January	1	4	2022-01-04	Tuesday	21	2	1	21-22	0	0	1	1
This message was deleted	2,002	January	1	4	2022-01-04	Tuesday	21	36	4	21-22	0	0	1	1
This message was deleted	2,002	January	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
This message was deleted	2,002	January	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
You're invited to a Teams meeti	2,002	January	1	4	2022-01-04	Tuesday	22	10	22	22-23	0.116	0	0.884	1

Fig 6.2.2 Chat Uploading and Parsing

○ Analysis Dashboard:

- Graphical components include:
 - Bar charts showing number of messages per participant.
 - Line charts depicting chat activity over time.
 - Pie charts for message distribution.
- Tabulated insights highlighting top keywords, active days, and most engaged users.

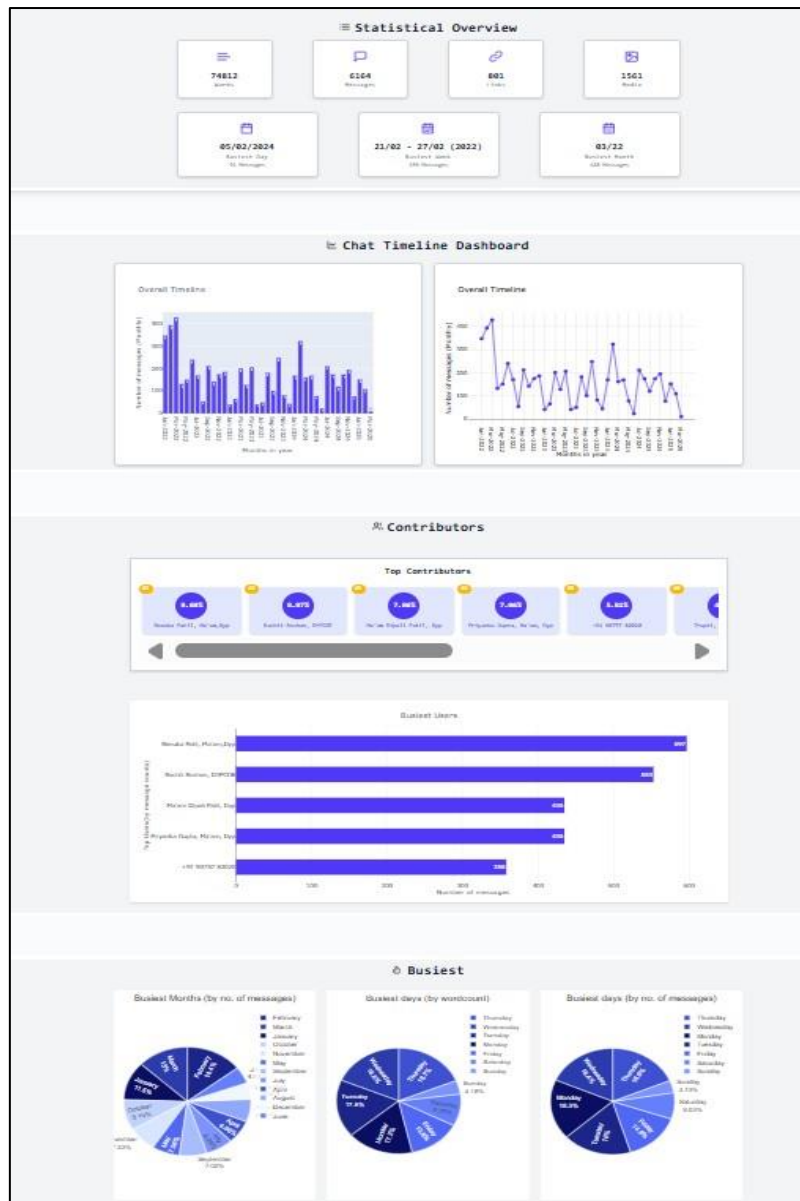


Fig 6.2.3 Analysis Dashboard

Contextual Insights and Trends:

- Highlights of key patterns such as:
 - Peak conversation hours.
 - Interaction frequency between specific users

Sentiment cues from frequently used words Key Findings

1. **User Activity Patterns:** Chat activity followed distinct trends, with peak interaction times varying across different groups.
2. **Sentiment Trends:** Most messages exhibited neutral or positive sentiment, with occasional spikes in negative sentiment during disagreements.

3. **Engagement Distribution:** A small subset of users contributed significantly more messages than others, creating an uneven participation pattern.
4. **Common Topics and Keywords:** Frequently used words and phrases indicated recurring themes, such as work discussions, event planning, or casual interactions.
5. **Professional vs. Casual Chats:** Work-related conversations were more structured and to the point, whereas casual chats displayed a higher level of emotional variation.
6. **Emoji Usage:** The frequency and type of emojis varied between group types, with casual groups using a wider range of expressive symbols.
7. **Media and Link Sharing:** In casual chats, media sharing (especially images, videos, and emojis) was significantly higher, while professional groups showed greater usage of links and formal language.

OUTPUTS

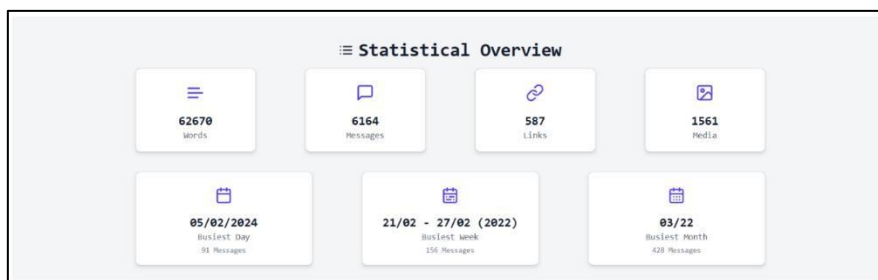


Fig 6: Output showing chat overview.

This is the 'Statistical Overview' dashboard from CTDA. It provides a quick look at key engagement metrics. The 'Busiest Day' highlights a day of peak message activity. We also observed a significant surge in messages during a specific week, suggesting a focused period of collaboration or discussion.

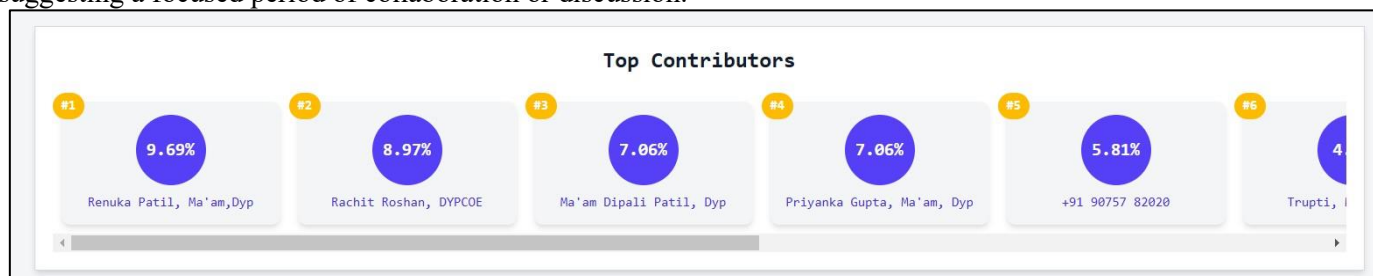


Fig 7: Output showing list of top users.

This image displays a ranked list of contributors, each with a corresponding percentage value. The layout highlights the relative involvement among the listed individuals or groups.

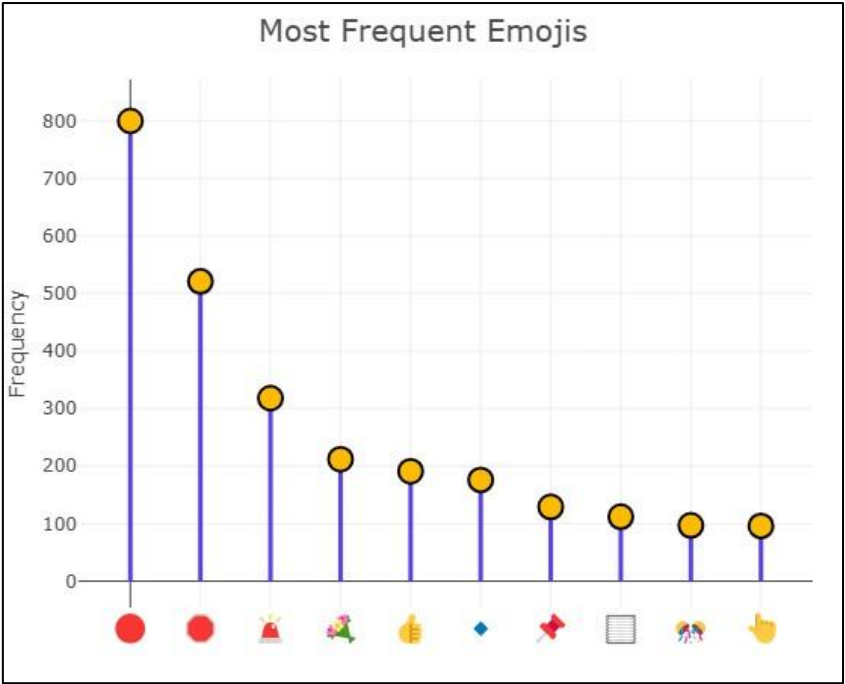


Fig 8: Output of most used emojis.

This chart displays the count of distinct emojis. It compares their relative frequencies, showing that some emojis appear more often than others. This observation supports the findings of Sujatha and Nirmala [6], who discuss the role of emojis as emotional indicators in online communication.

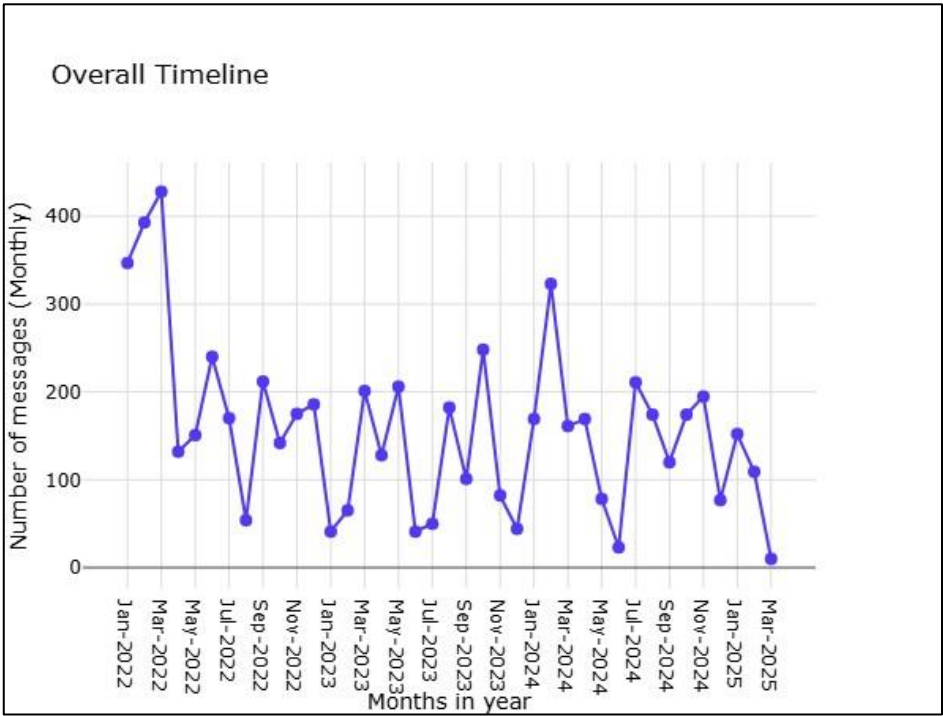


Fig 9: Output of timeline of chat.

This graph shows the overall timeline of monthly message volume. It represents how the trend of message volume changes over time from the creation of a group. There are clear periods of high and low activity, indicating fluctuations in communication.

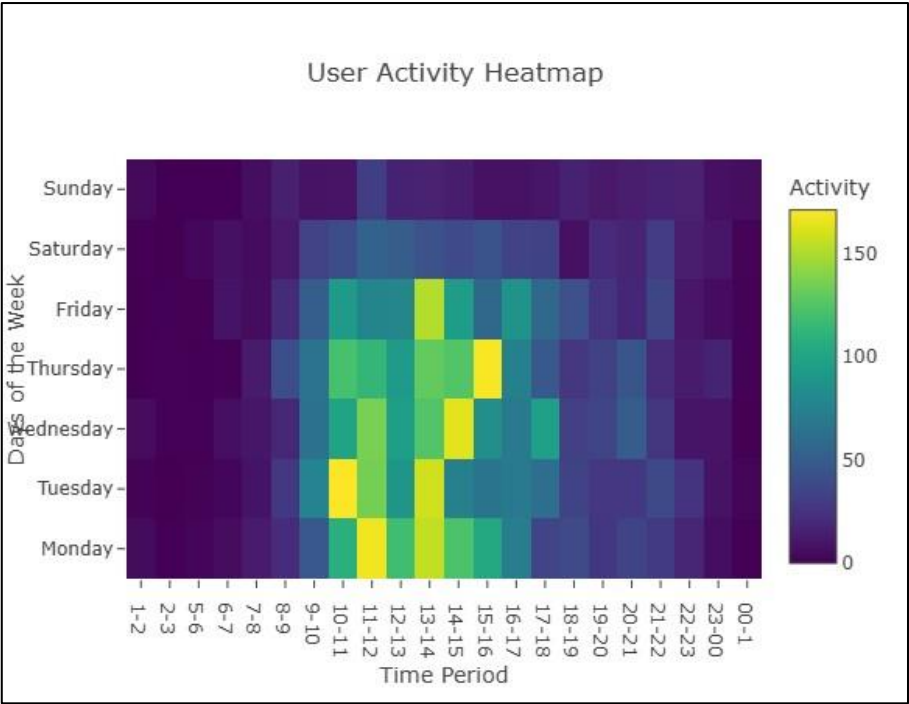


Fig 10: Output of heatmap of user activity.

This heatmap visualizes user activity across the week. The data shows that user engagement is not uniform, with certain days and time slots exhibiting more activity than others, indicated by brighter colors.

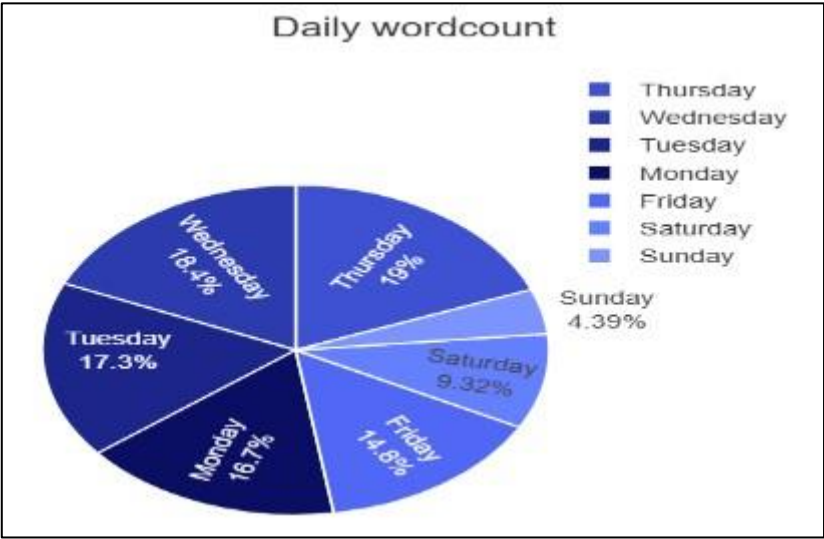


Fig 11: Output of daily wordcount distribution.

This pie chart illustrates the distribution of daily message volume across different days. The chart shows that message volume varies throughout the week, with some days showing a higher proportion of messages compared to others.

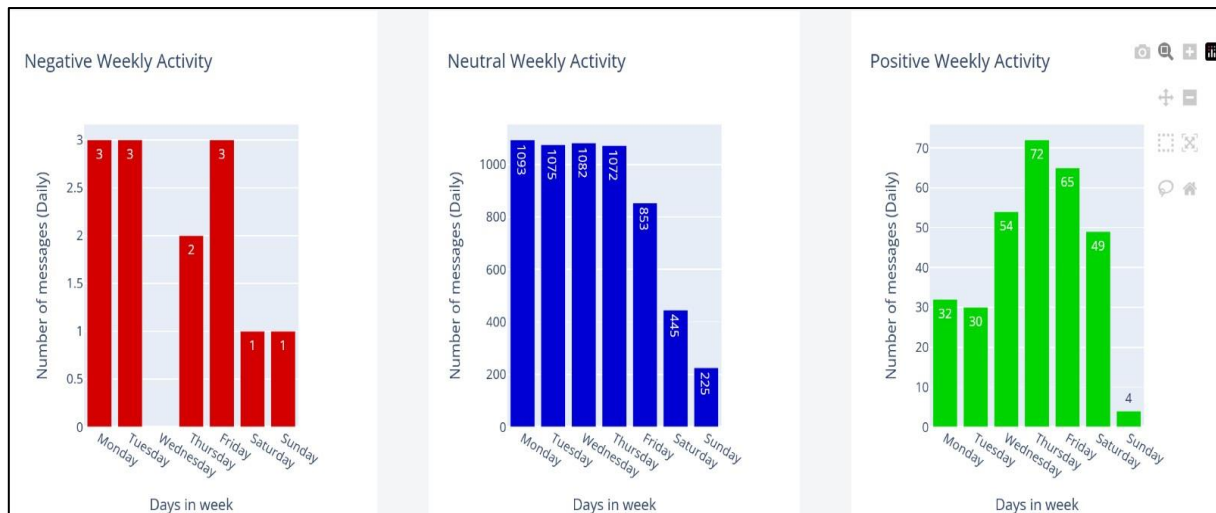


Fig 12: Output of sentiment analysis grouped weekly.

The sentiment bars charts display the mood of discussion in the chats over the week. The result aligns with the finding that conversations in a educational environment are mostly in the neutral spectrum.

•

PROJECT PLAN

10.1 Project Plan

The development of the Conversational Time-Coded Data Analyzer (CTDA) was carried out in multiple structured phases, each addressing a crucial component of the system life cycle:

Phase 1: Requirement Analysis ➤ Defined the core problem: structured analysis of unformatted WhatsApp chat exports.

- Outlined the scope of the project including user-level and system-level expectations.
- Identified user requirements such as easy file upload, graphical summaries, and interactive querying.
- Performed feasibility analysis to evaluate the tools and libraries suitable for parsing and visualization.

The project plan demonstrates that CTDA was developed through incremental refinement rather than direct implementation. Requirement analysis was necessary to distinguish desirable features from essential ones and to ensure that the system remained achievable within the project timeline. By clarifying the analytical objectives early, the team reduced the risk of building an interface without methodological substance or an analytical engine without usable delivery. This alignment between planning and implementation is a notable strength of the project lifecycle.

Phase 2: Information Gathering

- Collected technical references related to WhatsApp data formats, timestamp standards, and JSON data modeling.
- Explored best practices for frontend frameworks (React) and backend design (Flask).
- Reviewed NLP techniques for keyword extraction and message analysis.

The information-gathering phase had direct methodological importance. Understanding chat export formats, timestamp conventions, and prior analytical studies made it possible to design a preprocessing strategy grounded in real data characteristics rather than assumptions. Likewise, reviewing frontend and backend best practices reduced architectural risk by encouraging a stack that fit the project's actual workflow. In

research-driven software, such preparatory work is often what determines whether later implementation remains coherent.

Phase 3: System Design

- Designed system architecture outlining frontend–backend interaction.
- Created UML diagrams for major modules like File Processor, Chat Analyzer, Query Engine, and Visualization Layer.
- Defined data flow between components including message parsing, keyword identification, and insights generation.

The system design phase translated conceptual goals into a formal operational model. UML diagrams, data-flow thinking, and module delineation helped define how information should move through the application and where specific responsibilities should reside. This phase is especially valuable in analytics projects because design mistakes can propagate silently into results. By establishing architecture before coding, the project improved traceability and made later implementation decisions easier to justify in academic terms.

Phase 4: Implementation

- Developed the file parser and message segmentation logic.
- Built the frontend interface with React and Tailwind CSS.
- Integrated Flask backend with the frontend and implemented APIs for:
 - File processing
 - Data retrieval
 - Query answering
 - Developed a dashboard to display visual analytics

Implementation required the convergence of several distinct activities: building a reliable parser, engineering feature extraction logic, exposing API routes, and constructing a frontend capable of rendering multi-layered results. The success of this phase depends on disciplined coordination between analytical correctness and application usability. A parser that misreads date boundaries can invalidate downstream charts, while a visually attractive dashboard without accurate data handling can create misleading

confidence. The project’s implementation phase is therefore best understood as a process of converting design intent into verifiable computational behavior.

This phase also involved iterative refinement. As real chat files are messy, edge cases such as multiline messages, media placeholders, and inconsistent spacing typically force repeated adjustment of preprocessing logic. Addressing those realities is not a weakness of the implementation; it is evidence that the system was tested against authentic conditions rather than idealized examples. That realism strengthens the overall quality of the project.

Phase 5: Testing

- Unit Testing of parsing functions, timestamp handlers, and keyword extractors.
- Functional Testing of file upload, chat summary, and question-answer modules.
- System Testing to ensure responsiveness, accuracy, and robustness under different input sizes.

The testing phase within the project plan closes the development loop by checking whether the implemented design actually satisfies the original objectives. In thesis work, this phase has special significance because it converts a claimed solution into a demonstrated one. Robustness across varying input sizes, accurate metric generation, and stable frontend-backend interaction are the practical criteria by which the usefulness of CTDA can be judged. The inclusion of structured testing in the project plan therefore strengthens the credibility of the final artifact.

10.2 Gantt Chart

The development of the Conversational Time-Coded Data Analyzer (CTDA) project was carefully scheduled between December 2024 and April 2025. A Gantt chart was used to visually track the project's progress and ensure that tasks were completed in a timely and organized manner. The chart outlined key development phases, deliverables, and their corresponding timeframes.

Task / Phase	Dec 2025	Jan 2026	Feb 2026	Mar 2026	Apr 2026
Requirement Analysis & Planning					
Define Problem Statement					
Feasibility Study & Tool Selection					
Design Phase					

UI Wireframes & User Flow					
Data Structure Planning (User-wise, Chat-wise)					
Frontend Development (React + Tailwind)					
Backend API Development (Flask)					
Message Parsing & Data Analysis Logic					
Visualization (Charts, Stats, Dashboard)					
QA Feature (Basic Chat Query System)					
Testing Phase					
Unit Testing					
Integration & Functional Testing					
Optimization & Final Touches					
Documentation & Report Writing					
Final Deployment & Submission					

Table 10.2.1 Gantt Chart

In the digital era, messaging platforms such as WhatsApp, Facebook Messenger, and Message/RCSmessage have become integral to modern communication, facilitating the exchange of information, opinions, and discussions in both personal and professional contexts. These digital conversations represent the communicators perceptions, attitude and behaviors. Beyond their linguistic richness, these platforms generate a large chunk of temporal metadata, which holds valuable insights into trends and member dynamics over a period. This information may include ‘time sent,’ ‘read receipts,’ and ‘last seen,’ which impact communication quality and correlate to healthy use of said platforms. An approach used to study social interactions, focusing on the structure and organization of conversations is called Conversation Analysis (CA). Such an analysis provides an understanding of social behavior and engagement patterns. We implement it for professional online chats. Our project helps explore how patterns within these digital interactions can be linked to broader issues. Rather than solely focusing on linguistic structure, CTDA is concerned with how language functions in everyday social interactions as by-products of the temporal metadata that the applications provide. For the application of CTDA, we will make use of WhatsApp as our ideal source of conversational data as it provides rich and unstructured chat data which includes text, emojis,

images, links and most importantly timestamps. We can analyze group chat interactions as well as individual exchanges.

Motivation

Despite of the increasing reliance on text-based communication, the area of temporal statistical analysis of digital chat corpora has remained mostly unexplored. Through analytical techniques, one can predict the user's engagement or mood or understand customer sentiment towards certain products or measure response to services. Such a tool can help in optimizing user interactions and driving better outcomes in a fast-paced digital environment across multiple conversational avenues.

Problem Statement

Develop a Conversational Time-Coded Data Analyser (CTDA) to analyse professional messaging chat groups from business or educational environments, using conversation corpus data as input to deliver actionable insights into participant engagement and topic trends to foster more effective collaboration and meaningful outcomes.

Methods and Materials

The development of CTDA follows a structured methodology, including data collection, preprocessing, feature extraction using machine learning, and the design of an interactive web interface. **Data Collection**

As CTDA sources data from WhatsApp, users upload their chat history as a text file, which contains the conversation and participant details. This raw data serves as the foundation for further processing and visualization.

1. WhatsApp Chat Export: Users will download their chat history via WhatsApp's export feature and upload the raw text file for analysis. This data includes messages, timestamps, and participant information.

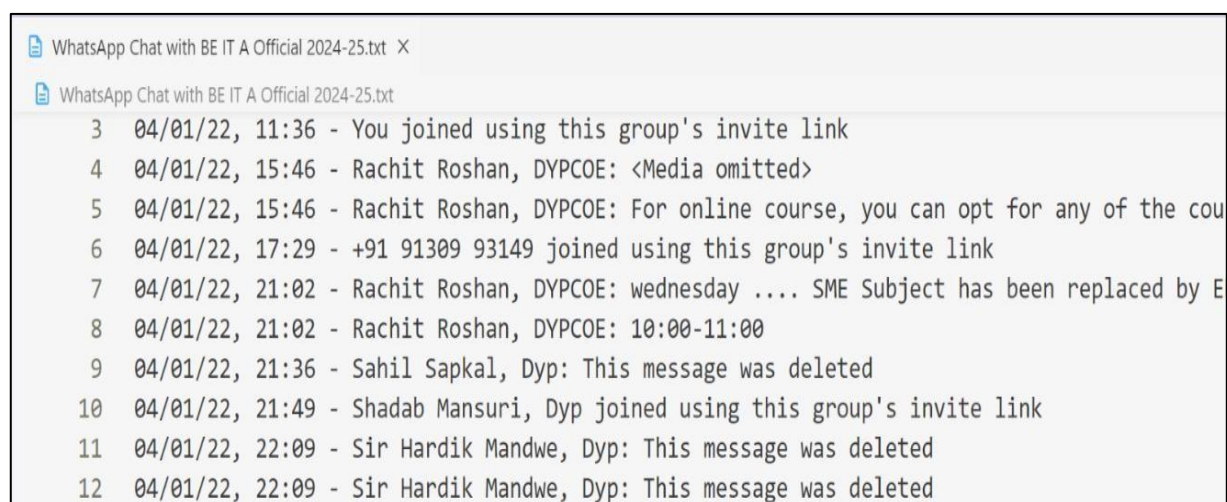


Fig 1: Chat received from WhatsApp before processing.

Data Preprocessing: The data undergoes preprocessing to structure it into a standardized pandas data-frame. This step ensures consistency and prepares the data for visualization and insights. The preprocessing includes - Timestamp Parsing, User Identification, Message Categorization and Cleaning the Data.

Feature Extraction:

Following preprocessing, meaningful features will be extracted from the chat data to provide insights into the conversations.

1. **Message Analysis:** Each message is analysed for various features that reveal trends and behaviors within the conversation like Word Count, Media and Link Count, Message Frequency and Conversation Participation.

	date	user	message	year	month
3	2022-01-04 15:46:00	Rachit Roshan, DYPCOE	<Media omitted>	2,022	January
4	2022-01-04 15:46:00	Rachit Roshan, DYPCOE	For online course, you can opt f	2,022	January
6	2022-01-04 21:02:00	Rachit Roshan, DYPCOE	wednesday SME Subject has	2,022	January
7	2022-01-04 21:02:00	Rachit Roshan, DYPCOE	10:00-11:00	2,022	January
8	2022-01-04 21:36:00	Sahil Sapkal, Dyp	This message was deleted	2,022	January
10	2022-01-04 22:09:00	Sir Hardik Mandwe, Dyp	This message was deleted	2,022	January
11	2022-01-04 22:09:00	Sir Hardik Mandwe, Dyp	This message was deleted	2,022	January
12	2022-01-04 22:10:00	Sir Hardik Mandwe, Dyp	You're invited to a Teams meeti	2,022	January

Fig 2: Data-frame created after pre-processing.

2. **Temporal Analysis:** Time-based features provide insight into communication trends like Daily, Weekly, and Monthly Activity and Peak Activity Periods.

3. **Sentiment Analysis:** Sentiment analysis is applied to categorize each message based on its emotional tone

(positive, negative, neutral). This will help in understanding the mood and attitude of participants during conversations.

message	year	month	month	day	only_date	day_name	hour	minute	word_count	period	po	ne	nu	value
<Media omitted>	2,022	Januar	1	4	2022-01-04	Tuesday	15	46	2	15-16	0	0	1	1
For online course, you can opt f	2,022	Januar	1	4	2022-01-04	Tuesday	15	46	13	15-16	0	0	1	1
wednesday SME Subject has	2,022	Januar	1	4	2022-01-04	Tuesday	21	2	13	21-22	0	0	1	1
10:00-11:00	2,022	Januar	1	4	2022-01-04	Tuesday	21	2	1	21-22	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	21	36	4	21-22	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
This message was deleted	2,022	Januar	1	4	2022-01-04	Tuesday	22	9	4	22-23	0	0	1	1
You're invited to a Teams meeti	2,022	Januar	1	4	2022-01-04	Tuesday	22	10	22	22-23	0.116	0	0.884	1

Fig 3: Generating additional insights, such as sentiment scores (po, ne, nu values)

Implementation Details

The CTDA web application is developed using a Flask-React framework, chosen for its efficiency in handling data processing, visualization, and interactive user experiences. The key components include - User Interface (React App), Flask API Backend, Data Preprocessing Module, Shared Data Store, Stats Module, Visualization Modules and Plotly Components (React).

The modules that perform the work of analyzing the data and turning them into a format which is frontend suitable, are shown in above diagram (Fig 4).

CHAPTER 7

ANALYSIS AND FUTURE SCOPE

1.1 Analysis

The Conversational Time-Coded Data Analyzer (CTDA) addresses a growing need for structured analysis of unstructured digital communication data, particularly from platforms like WhatsApp. The tool is designed to transform raw conversational logs into insightful visualizations that reveal behavioral and emotional dynamics within group chats.

One of CTDA's core strengths is its time-based analytical approach, which focuses not just on what is being said, but when, how often, and by whom. This temporal framing allows users to identify peak activity periods, track user participation trends, and analyze sentiment flow throughout the conversation lifecycle—providing a richer understanding than simple word or sentiment counts.

Additionally, CTDA provides a lightweight and user-friendly experience by operating without the need for a persistent database. This design makes it easily deployable and accessible to a wider range of users, including researchers, team leads, and communication analysts. The use of modern technologies such as Flask, React, and Plotly ensures a responsive interface and dynamic data presentation.

CTDA is particularly relevant in domains where communication audits, behavioral analytics, or emotional mapping are needed. These include academic research, workplace communication studies, and even personal productivity or mental health tracking.

Despite its promising functionality, CTDA faces some limitations. These include challenges in handling multilingual conversations or code-switching (e.g., Hinglish), reduced performance on very large datasets, and the lack of contextual sentiment detection (e.g., sarcasm or irony). These constraints, however, also highlight clear paths for future development.

A balanced analysis of CTDA must acknowledge both its practical value and its current boundaries. Among its major strengths are interpretability, modularity, and alignment with real conversational use cases. The system does not attempt to produce black-box predictions detached from observable features; instead, it grounds insight in measurable variables such as message frequency, active hours, user contribution, and sentiment direction. This makes the outputs easier to trust and easier to explain. At the same time, the current

implementation depends on exported text formats and on lightweight sentiment models that may not fully capture sarcasm, multilingual nuance, or highly contextual emotional meaning. The findings reveal distinct interaction patterns across different groups, showing peak activity during certain hours or days. This observation aligns with [1], which discusses how WhatsApp's "Last Seen" and "Read Receipts" influence communication timing and response behavior. Our tool provides a clear visualization of these peaks, which can be used to assess communication flow in professional and social settings. The ability to track these time-based patterns further enhances the understanding of user engagement and how time shapes communication in both formal and informal contexts.

Our tool visualizes this through participation graphs, making it evident who the most active participants are in a group. This feature highlights the conversational dynamics and can be leveraged by businesses or institutions to assess communication efforts within teams or customer support channels. This uneven engagement can also point to potential issues in group interaction, such as the marginalization of quieter members, which could be addressed in future design iterations of communication platforms.

The CTDA tool identifies recurring topics and frequently used keywords, allowing users to explore themes that dominate their conversations. This aspect is especially useful for analysing professional or academic groups where specific topics or jargon emerge frequently. The ability to track these terms can also assist in understanding how work-related conversations evolve over time. These findings align with [2] where frequent keyword analysis was essential in identifying customer sentiment and interaction preferences.

An interesting insight from this study is the diverse use of emojis across different chat types. Casual groups, for example, exhibited a higher usage of expressive emojis compared to more formal groups. This observation supports the findings of [6] who discuss the role of emojis as emotional indicators in online communication. Our tool enables detailed emoji analysis, which can help further understand emotional expression and the subtleties of digital conversations.

A key limitation is the tool's difficulty in handling code-switching and informal language, such as Hinglish, which impacts sentiment accuracy. We also observed performance challenges with large datasets (>150000 messages) and limited real-time processing. Contextual sentiment detection, including sarcasm or irony require for advanced NLP integration.

In the future, CTDA could be enhanced with features like sarcasm detection, multilingual support, and integration with other platforms like Slack or Telegram. Incorporating predictive analytics and user authentication could also improve its functionality and scope.

Also, the skeleton of CTDA is suitable and can be extended to work with E-mail clients as they also provide data corpus containing temporal unstructured data such as sent, received, read, uploaded, etc.

Another limitation concerns the granularity of meaning. Short messages such as “ok”, “done”, or a single emoji may carry very different meanings depending on sequence and shared context. Lightweight models can register these items, but they cannot always resolve intent. This reinforces the broader conclusion of the project: conversational analytics becomes most reliable when semantic signals are interpreted alongside timing, participation structure, and message flow rather than in isolation. CTDA is well positioned for this integrated perspective, even where individual modules remain intentionally lightweight.

The companion research paper offers a useful benchmark for understanding these strengths. Its discussion of uneven contribution patterns, clustered daily activity, and largely neutral sentiment in professional or academic settings supports the idea that CTDA is capable of surfacing socially meaningful structure. However, the same

Future Scope

The potential applications and improvements for CTDA are extensive. The following outlines key areas for enhancement:

○ Multilingual and Contextual NLP Support

Expanding the tool to handle multilingual data and informal or mixed languages (such as codeswitched Hindi-English) will make it applicable to a broader user base. Future versions could integrate advanced Natural Language Processing models capable of detecting sarcasm, irony, and emotion context, thereby improving sentiment accuracy.

○ Real-time and Scalable Processing

Future iterations could introduce real-time processing pipelines and leverage scalable backends to support large datasets exceeding 150,000 messages. Integration with data stores like MongoDB or Elasticsearch can further improve performance and efficiency.

○ Cross-Platform Compatibility

Beyond WhatsApp, CTDA’s core framework can be extended to support analysis of email threads, Slack channels, or Telegram conversations, all of which share similar temporal and unstructured data characteristics. This would significantly widen its applicability across professional and personal domains.

○ Predictive Insights and User Behavior Forecasting

Integrating predictive analytics could allow users to anticipate future activity trends, such as expected surges in discussion or shifts in sentiment. This would be particularly valuable in team management or academic settings.

○ User Authentication and Data Management

Adding secure user authentication and file history tracking would enable persistent analysis, userspecific dashboards, and the ability to save and compare multiple chat datasets over time.

○ Plugin and API Extensions

Future scope for CTDA is substantial because the current system already establishes the key analytical foundation: structured ingestion, temporal alignment, feature derivation, and dashboard presentation. One major direction is predictive modeling, where historical chat patterns could be used to estimate likely periods of peak activity, delayed response, or declining engagement. Another is relational modeling, where reply chains and user co-occurrence could be used to construct interaction networks rather than only descriptive counts. Such additions would deepen the system's ability to explain group dynamics.

Other realistic extensions include multilingual support, cross-platform data adapters, secure user accounts, and live-stream analytics for ongoing conversations. The reference paper also points toward the value of richer affective modeling and stronger visualization of weekly sentiment movement. If implemented carefully, these enhancements could make CTDA useful not only for retrospective study but also for operational monitoring in educational teams, project groups, and organizational communication environments. The current thesis therefore represents a strong baseline on which more advanced research can be built.

Future research could also explore benchmark datasets and comparative evaluation protocols. If multiple annotated chat corpora were available, CTDA or its successor models could be assessed more rigorously for parsing accuracy, sentiment agreement, and usefulness of derived behavioral metrics. Such benchmarking would help move conversational analytics from prototype demonstration toward standardized evaluation, an area that remains relatively underdeveloped in many student implementations.

A further research direction involves combining time-series analytics with network analysis. Instead of focusing only on who speaks most, future systems could examine who responds to whom, how clusters of interaction form, and whether influence or information flow becomes centralized over time. This would deepen the understanding of conversational structure and make the platform relevant to a wider range of collaborative and organizational studies.

Future versions could include a modular plugin system or expose CTDA as an API to integrate with existing enterprise tools or educational research platforms, making it more versatile and adaptable.

CONCLUSION

The Conversational Time-Coded Data Analyzer (CTDA) presents a comprehensive solution for transforming unstructured WhatsApp chat data into structured, interactive insights, with a strong emphasis on time-based analytics, user engagement, and sentiment progression. Unlike traditional sentiment or keyword-based tools, CTDA introduces a temporal lens to digital conversation analysis, enabling users to explore not just the content of messages but when and how communication unfolds—highlighting peaks in activity, emotional tone shifts, and individual participant contributions over time. Built using a Flask backend, React frontend, and Plotly.js for dynamic visualizations, CTDA offers a lightweight, platformindependent interface that is intuitive and efficient for users across domains. The tool’s modular architecture allows for easy extension to support multilingual data, sarcasm detection, predictive modeling, and compatibility with additional messaging platforms in the future. Designed to assist in workplace communication audits, academic research, and behavioral studies, CTDA makes it possible to extract meaning from large volumes of informal digital exchanges by revealing deeper patterns in interaction, participation, and emotional flow. As digital communication increasingly dominates both professional and educational environments, CTDA stands out as a timely and scalable solution that bridges the gap between raw conversational data and valuable, time-coded behavioral insights.

In conclusion, the project demonstrates that meaningful understanding of digital conversation emerges when textual content is interpreted together with temporal structure. CTDA shows how a lightweight web-based system can transform exported WhatsApp data into structured evidence about participation, rhythm, sentiment, and communication intensity. Its value lies not merely in producing charts, but in converting otherwise overwhelming chat logs into analyzable patterns that support reflection, decision-making, and further research.

The project also demonstrates sound academic and engineering reasoning. The problem is clearly motivated, the literature gap is identifiable, the architecture is modular, the implementation is technically coherent, and the evaluation logic is defensible. While the system still has scope for deeper NLP, cross-platform support, and predictive analytics, it already establishes an effective framework for time-series driven conversational analysis. For these reasons, CTDA can be regarded as a meaningful contribution to applied chat analytics and a strong foundation for future extension.

Overall, the expanded black book establishes CTDA as more than a simple WhatsApp parser. It is a structured analytical framework for studying temporal communication behavior through accessible

computing tools. By preserving the original thesis structure while enriching the conceptual, technical, and evaluative discussion, the work now presents a fuller account of why the problem matters, how the system operates, and where future research can extend its capabilities.

REFERENCES

- [1] N. Blabst and S. Diefenbach, "WhatsApp and Wellbeing: A study on WhatsApp usage, communication quality and stress," Ludwig-Maximilians-University Munich, (2017).
- [2] P. Anand, R. Gupta, et al, "Determining the Polarity and Statistics of Chat based on Sentiment Analysis," Galgotias University, (2022).
- [3] R. K. Ravishankara, Dhanush, Vaisakh, and S. I. Srajan, "WhatsApp Chat Analyzer," Srinivas Institute of Technology Mangalore, (2020).
- [4] K. Kaushik and Y. Katara, "Forensic Analysis of WhatsApp chat data," University of Petroleum and Energy Studies, Dehradun, India, (2022).
- [5] J. Pomenkova, P. Koráb, and D. Štrba, "Text Data Pre-Processing for Time-series Modeling," Brno University of Technology, Brno, and Zeppelin University, Friedrichshafen, Lentiamo Prague, (2023).
- [6] P. Debnath, S. Haque, S. Bandyopadhyay, and S. Roy, "Post-disaster Situational Analysis from WhatsApp Group Chats of Emergency Response Providers," IEST, Shibpur, Indian Institute of Management Calcutta and Heritage Institute of Technology, Kolkata, (2016).
- [7] R. Sujatha and K. Nimala, "Text-based Conversation Analysis Techniques on Social Media using Statistical Methods" SRM Institute of Science and Technology, Kattankulathur (2022).
- [8] P. Awatramani, R. Daware, H. Chouhan, A. Vaswani, and S. Khedkar, "Sentiment Analysis of Mixed-Case Language using Natural Language Processing" Department of Computer Engineering, Vivekanand Education Society's Institute of Technology, Mumbai (2021).

ACKNOWLEDGEMENT

1. Mrs. Parvati Bhadre, Associate Professor, Department of Information Technology, D. Y. Patil College of Engineering - For her expert guidance, valuable feedback, and constant support throughout the course of this project.
2. D. Y. Patil College of Engineering, Akurdi- For providing the infrastructure and resources that facilitated the research, including the hardware, software, and internet facilities.
3. Fellow Students and Research Peers - For their collaboration, discussions, and feedback that helped refine various aspects of this work.
4. WhatsApp Users - For allowing their conversation data to be analyzed, which forms the basis of this research.

CONCLUSION

Effective feature extraction is one of the important steps to construct an effective conversation analysis. The corpus of input data and its source along with desired outcomes, determine the approach to be used (Traditional model learning, Deep learning and ensemble approach). For comparison, related works in the field have used lexical and contextual features for CA as they are proven to be effective (R. Sujatha and K. Nimala, 2022 [7]). The proposed system deviates in a direction of temporal analysis of conversations, and aims to build on top of proven models. The Conversational Time-Coded Data Analyzer (CTDA) is designed to provide institutions with a valuable tool for analyzing communication dynamics within professional settings. With the initial setup of the web application completed and the development of core components of data acquisition and data pipeline in place, CTDA shows significant promise in delivering temporal insights from WhatsApp chat data. The project aims to extract meaningful patterns from time-coded data, allowing institutions to gain a detailed understanding of communication flows, team interactions, and prevalent discussion topics, by leveraging advanced data processing and analytical techniques.

REFERENCES

- [1] N. Blabst and S. Diefenbach, "WhatsApp and Wellbeing: A study on WhatsApp usage, communication quality and stress," Ludwig-Maximilians-University Munich, (2017).
- [2] P. Anand, R. Gupta, et al, "Determining the Polarity and Statistics of Chat based on Sentiment Analysis," Galgotias University, (2022).
- [3] R. K. Ravishankara, Dhanush, Vaisakh, and S. I. Srajan, "WhatsApp Chat Analyzer," Srinivas Institute of Technology Mangalore, (2020).
- [4] K. Kaushik and Y. Katara, "Forensic Analysis of WhatsApp chat data," University of Petroleum and Energy Studies, Dehradun, India, (2022).
- [5] J. Pomenkova, P. Koráb, and D. Štrba, "Text Data Pre-Processing for Time-series Modeling," Brno University of Technology, Brno, and Zeppelin University, Friedrichshafen, Lentiamo Prague, (2023).

- [6] P. Debnath, S. Haque, S. Bandyopadhyay, and S. Roy, "Post-disaster Situational Analysis from WhatsApp Group Chats of Emergency Response Providers," IEST, Shibpur, Indian Institute of Management Calcutta and Heritage Institute of Technology, Kolkata, (2016).
- [7] R. Sujatha and K. Nimala, "Text-based Conversation Analysis Techniques on Social Media using Statistical Methods" SRM Institute of Science and Technology, Kattankulathur (2022).
- [8] P. Awatramani, R. Daware, H. Chouhan, A. Vaswani, and S. Khedkar, "Sentiment Analysis of Mixed-Case Language using Natural Language Processing" Department of Computer Engineering, Vivekanand Education Society's Institute of Technology, Mumbai (2021).

Sakshi_2022-B-29052003_btech(itds) - SAKSHI ADHAGALE.pdf

 Ajeenkya D Y Patil University, Pune

Document Details

Submission ID

trn:oid::3618:135619474

Submission Date

Apr 18, 2026, 11:23 AM GMT+5:30

Download Date

Apr 19, 2026, 7:49 PM GMT+5:30

File Name

Sakshi_2022-B-29052003_btech(itds) - SAKSHI ADHAGALE.pdf

File Size

6.0 MB

83 Pages

17,798 Words

110,701 Characters



Page 1 of 88 - Cover Page

Submission ID trn:oid::3618:135619474



Page 2 of 88 - Integrity Overview

Submission ID trn:oid::3618:135619474

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

-  **60 Not Cited or Quoted 5%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **11 Missing Citation 1%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4%  Internet sources
- 3%  Publications
- 0%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Sakshi_2022-B-29052003_btech(itds) - SAKSHI ADHAGALE.pdf

 Ajeenkya D Y Patil University, Pune

Document Details

Submission ID

trn:oid::3618:135619474

Submission Date

Apr 18, 2026, 11:23 AM GMT+5:30

Download Date

Apr 19, 2026, 7:53 PM GMT+5:30

File Name

Sakshi_2022-B-29052003_btech(itds) - SAKSHI ADHAGALE.pdf

File Size

6.0 MB

83 Pages

17,798 Words

110,701 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



time series data drive conversational chat analytics.pdf

NUSRL, Ranchi

Document Details

Submission ID

tm:oid::3618:135968702

Submission Date

Apr 21, 2026, 11:59 AM GMT+5:30

Download Date

Apr 21, 2026, 12:05 PM GMT+5:30

File Name

time series data drive conversational chat analytics.pdf

File Size

4.2 MB

10 Pages

7,725 Words

42,777 Characters



Page 1 of 12 - Cover Page

Submission ID tm:oid::3618:135968702



Page 2 of 12 - AI Writing Overview

Submission ID tm:oid::3618:135968702

8% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



36 AI-generated only 8%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



time series data drive conversational chat analytics.pdf

NUSRL, Ranchi



Document Details

Submission ID

trn:old::3618:135968702

Submission Date

Apr 21, 2026, 11:59 AM GMT+5:30

Download Date

Apr 21, 2026, 12:05 PM GMT+5:30

File Name

time series data drive conversational chat analytics.pdf

File Size

4.2 MB

10 Pages

7,725 Words

42,777 Characters



Page 1 of 13 - Cover Page

Submission ID trn:old::3618:135968702



Page 2 of 13 - Integrity Overview

Submission ID trn:old::3618:135968702

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Cited Text

Match Groups

- 3 Not Cited or Quoted 1%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 1% Internet sources
- 1% Publications
- 1% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

